

Análisis de Sistemas. Objetos.

A continuación, se incluye el material de los Capítulos 7, 8, 9, 10, 11, y 18 de Arlow y Neustadt (2006).

Rumbaugh y colab. (2007) definen un objeto como una entidad discreta con un límite bien definido que encapsula el estado y comportamiento; una instancia de una clase. Puede pensar en un objeto como un paquete cohesivo de datos y función. Por lo general, la única forma de llegar a la parte de datos de un objeto es invocar una de las funciones que pone disponible el objeto. Estas funciones se denominan operaciones. Ocultar la parte de datos de un objeto detrás de esta capa de operaciones se conoce como encapsulación u ocultamiento de datos.

La encapsulación no se aplica en UML ya que algunos lenguajes orientados a objetos no lo demandan (tales como C++, Java). Sin embargo, ocultar la parte de datos de un objeto detrás de su conjunto de operaciones siempre es un buen estilo orientado a objetos.

Todo objeto es una instancia de cierta clase que define el conjunto común de características (atributos y operaciones) que se comparten por todas las instancias de esa clase. La idea de clases y clasificadores es muy sencilla. Piense en una impresora del tipo “HP Color LaserJet CP 2025”. Esto describe las propiedades de todas las instancias específicas de esta clase, como la “HP Color LaserJet CP 2025 S/N 34120098” en particular en mi oficina. Una instancia específica de una clase se denomina objeto.

Pensando un poco más en este ejemplo, puede ver que tiene ciertas propiedades que son comunes a todos los objetos:

- **Identidad.** Ésta es la existencia única del objeto en tiempo y espacio. Es lo que le hace diferente de los otros objetos. En nuestro ejemplo, el número de serie de la impresora se puede utilizar como el identificador para apuntar a esta impresora en particular en nuestro escritorio y representa la identidad única de ese objeto. Un número de serie es una forma buena de especificar la identidad de un objeto físico, y nosotros utilizamos un principio similar, la idea de identificador de un objeto, para especificar la identidad de cada uno de los objetos de software con los que trabajaremos en el análisis y diseño orientado a objetos. Por supuesto, en el mundo real, no todos los objetos tienen un número de serie, pero tienen identidades únicas debido a sus coordenadas espaciales y temporales particulares. De forma similar, en los sistemas de software orientados a objeto todo objeto tiene algún tipo de identificador de objeto.
- **Estado.** Esto está determinado por los valores de atributo de un objeto y las relaciones que tiene el objeto con otros objetos en un punto en el tiempo en particular. La Tabla 1 proporciona una lista no exhaustiva de estados por los que puede pasar una impresora. Puede ver por esto cómo el estado de un objeto depende de los valores de sus atributos y su conexión con otros objetos.
- **Comportamiento.** Existen ciertas tareas que la impresora puede hacer por nosotros, tales como: `encender()`, `apagar()`, `imprimirDocumento()`, `alimentarHojaManualmente()`, `cambiarCartuchoDeTinta()`.

Invocar una operación sobre un objeto causará un cambio en los valores de uno o más de sus atributos o en sus relaciones con otros objetos, y esto puede constituir una transición de estado. Éste es un movimiento significativo del objeto de un estado a otro estado. Considerando la Tabla 1, es claro que el estado de un objeto también puede afectar su

comportamiento. Por ejemplo, si a su impresora la falta tinta (estado del objeto `NoTieneTintaNegra`), entonces invocar la operación `imprimirDocumento()` hará que muestre un error. El comportamiento actual de `imprimirDocumento()` depende del estado del objeto.

Una operación es la especificación de una pieza del comportamiento. Una implementación de ese comportamiento se denomina método.

Tabla 1. Estados de un objeto impresora

Estado de objeto	Atributo	Valor de atributo	Relación
Encendido	red	on	N/A
Apagado	red	off	N/A
NoTieneTintaNegra	cartuchoTintaNegra	vacío	N/A
NoTieneTintaColor	cartuchoTintaColor	vacío	N/A
Conectado	N/A	N/A	conectada a un objeto ordenador
NoConectado	N/A	N/A	no conectada a un objeto ordenador

Encapsulación

Todo objeto posee un identificador que lo hace único. En análisis orientado a objetos no tiene que preocuparse por cómo se implementa esta identidad; simplemente asume que cada objeto tiene una identidad única que se gestiona por la tecnología de implementación. En la Figura 1 se muestra una imagen conceptual de un objeto que enfatiza la encapsulación. La Figura 1 no es un diagrama UML. La Figura ilustra en el centro al estado del objeto, formado por un conjunto de valores de atributos (en este caso 12345678, "Nicolás", 300.00). Normalmente, algunos de estos valores de atributos permanecerán fijos, y algunos cambiarán con el tiempo. Por ejemplo, el número de cuenta permanecerá fijo, pero el saldo variará con el tiempo. A medida que el saldo cambia en el tiempo, vemos que el estado del objeto también cambia con el tiempo. Por ejemplo, si el saldo es negativo, entonces podemos decir que el objeto está en el estado Deudor. A medida que el saldo cambia de negativo a cero, el objeto realiza un cambio de estado Deudor a Vacío. Además, a medida que el saldo del objeto Cuenta se hace positivo, realiza otra transición de estado, desde Vacío a EnCrédito. Pueden existir más transiciones que éstas. En realidad, cualquier invocación de operación que lleve a un cambio sustantivo en la naturaleza del objeto crea una transición de estado. UML proporciona máquinas de estado para modelar cambios de estado.

El comportamiento de cualquier objeto es básicamente lo que puede hacer el objeto. En la Figura 1 se ilustran las operaciones listadas en la Tabla 2. Estas operaciones se ilustran en la figura como una cáscara que envuelve el estado del objeto. Este conjunto de operaciones especifica el comportamiento del objeto. Observe que invocar algunas de estas operaciones (depositar, retirar, o establecerPropietario) cambia los valores de atributos y puede generar transiciones de estado. La operación `obtenerPropietario()` no cambia ningún valor de atributo y por lo tanto no causará una transición de estado. La encapsulación es una de las ventajas

principales de la programación orientada a objetos y puede llevar a software más robusto y ampliable. En este sencillo ejemplo, un usuario de este objeto no tiene que preocuparse por la estructura de los datos ocultos dentro del objeto, sólo con lo que el objeto puede hacer; es decir, con los servicios que ofrece a otros objetos.

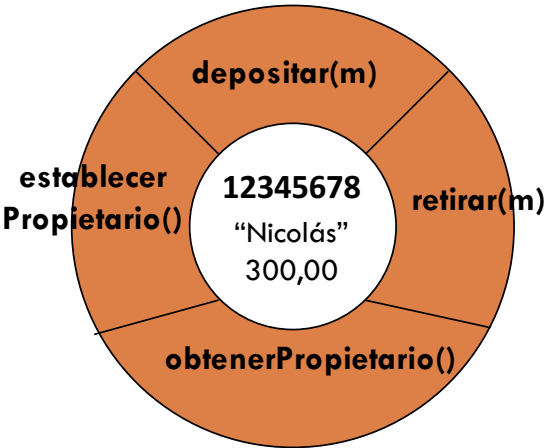


Figura 1. Encapsulación de un objeto.

Tabla 2.

Operación	Semántica
depositar(m)	Depositar la cantidad m en el objeto Cuenta. Incrementa el valor del atributo saldo.
retirar(m)	Retirar el dinero m de la Cuenta. Reduce el valor del atributo saldo.
obtenerPropietario()	Devuelve el propietario del objeto Cuenta; es una operación de consulta.
establecerPropietario()	Cambia el propietario del objeto Cuenta.

Mensajería

Los objetos tienen valores de atributo y comportamiento, pero ¿cómo agrupa objetos para crear sistemas de software?. Los objetos colaboran para realizar las funciones del sistema. Lo que esto significa es que los objetos forman vínculos a otros objetos y envían mensajes a lo largo de estos vínculos. Cuando un objeto recibe un mensaje (en la Figura 2, el objeto Cuenta recibe el mensaje retirar(150,00) desde un objeto banco), mira en su conjunto de operaciones para ver si existe una operación cuya signatura coincide con la del mensaje. Si existe, entonces invoca la operación (en la Figura 2, operación retirar(m)). Estas signaturas componen el nombre del mensaje (u operación), tipos de parámetros y valor de retorno.

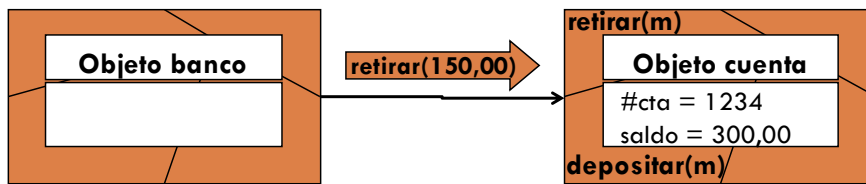


Figura 2. El objeto Banco envía el mensaje retirar 150,00 a un objeto Cuenta. El objeto Cuenta responde al invocar su operación retirar(real). Esta operación reduce el saldo de la cuenta en 150,00.

Notación de objeto UML

El icono de objeto UML es un rectángulo con dos compartimentos; en la Figura 3 se incluye un ejemplo. El compartimiento superior contiene el identificador de objeto, que siempre está subrayado. Esto es importante, ya que la notación UML para clases es muy similar a la de objetos.

@startuml

object "<u>cuentaDeNicolas:Cuenta</u>" as ctaNico {

numeroDeCuenta: String = "12345"

propietario: String = "Nicolás"

saldo: double = 300,00

}

@enduml

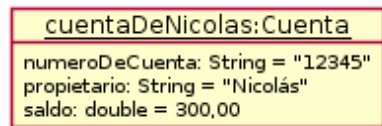


Figura 3. Definición del objeto cuentaDeNicolás, instancia de la Clase Cuenta

UML es muy flexible sobre cómo los objetos se pueden representar en diagramas de objeto. El identificador de objeto puede constar de cualquiera de las siguientes opciones:

- Sólo el nombre de la clase: Por ejemplo, :Cuenta (Figura 4). Esto significa que tiene un objeto anónimo, instancia de esa clase.
- Sólo el nombre de objeto: Por ejemplo, cuentaDeNicolás (Figura 4). Esto identifica un objeto específico pero no identifica a qué clase pertenece. Esta notación puede ser de utilidad en un análisis inicial cuando todavía no ha descubierto todas las clases.
- El nombre del objeto concatenado con el nombre de clase, separado por dos puntos (Figura 3). Puede leer los dos puntos como “una instancia de clase”. Por lo tanto, en la Figura 3 podría leer el diagrama de la siguiente forma: existe un objeto denominado cuentaDeNicolás que es una instancia de la clase Cuenta.

@startuml

object "<u>:Cuenta</u>" as unaCuenta {

```

}
object "<u>cuentaDeNicolas</u>" as ctaNico {
    numeroDeCuenta: String = "12345"
    propietario: String = "Nicolás"
    saldo: double = 300,00
}
@enduml

```

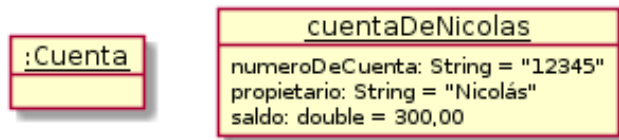


Figura 4

Los objetos se nombran normalmente en una combinación de mayúsculas y minúsculas, empezando con una letra minúscula. Los caracteres especiales, como espacios y subrayados, se evitan. Todos los objetos de la misma clase tienen exactamente el mismo conjunto de operaciones. Las operaciones se listan en el icono de clase, no en el icono de objeto. Los atributos se pueden mostrar opcionalmente en el compartimiento inferior del icono de objeto. Estos atributos que elige mostrar deben estar nombrados y pueden tener un tipo y valor opcional. Los atributos también se nombran poniendo en mayúscula la primera letra de cada palabra con la primera letra de todas en minúscula.

Valores de atributo de objeto

Todo valor de atributo de un objeto tiene la siguiente forma:

nombre: tipo = valor

Puede elegir mostrar todos, algunos o ningún valor de atributo de objeto dependiendo de la finalidad del diagrama.

Para mantener los diagramas de objeto sencillos y claros, puede elegir omitir tipos de atributo, ya que están definidos en la clase del objeto.

Clases

El manual de referencia de UML (Rumbaugh y colab., 2007) define una clase como el descriptor para un conjunto de objetos que comparten los mismos atributos, operaciones, métodos, relaciones y comportamiento. Resumiremos esto al decir que una clase es un descriptor para un conjunto de objetos que tienen las mismas características. Todo objeto es una instancia de exactamente una clase.

Piense en una clase como una plantilla para objetos: una clase determina la estructura (conjunto de características) de todos los objetos de esa clase. Todos los objetos de la misma clase deben tener el mismo conjunto de operaciones, el mismo conjunto de atributos y el mismo conjunto de relaciones, pero pueden tener diferentes valores de atributo.

Clasificador e instancia son una de las divisiones comunes de UML y el ejemplo más común de esta división es clase y objeto. Una clase es una especificación o plantilla que deben seguir todos los objetos de esa clase (instancias). Todo objeto de la clase tiene valores específicos

para los atributos definidos por la clase y responderán a los mensajes al invocar las operaciones definidas por la clase.

Dependiendo de su estado, diferentes objetos pueden responder al mismo mensaje de diferentes formas. Por ejemplo, si trata de retirar 100 \$ de un objeto cuenta bancaria que ya está en saldo deudor, esto puede dar un resultado diferente de tratar de retirar 100 \$ de un objeto cuenta bancaria que tiene varios cientos de \$ de crédito.

La clasificación es posiblemente la forma más importante que las personas tienen de ordenar información sobre el mundo. Como tal, es también uno de los conceptos orientados a objetos más importante. Utilizando la noción de clases puede hablar de un tipo de coche en particular o un tipo de árbol sin mencionar una instancia específica. Es lo mismo para el software. Las clases le permiten describir conjunto de características que todo objeto de la clase debe tener sin tener que cada uno de esos objetos. No hay una única manera de clasificar objetos en el mundo real. Dado que hay tantas opciones, elegir el esquema de clasificación más apropiado es uno de los aspectos más importantes del análisis y diseño orientado a objetos.

Existen relaciones entre clases, una clase es una subclase de otra, o, por el contrario, una clase es una superclase de otras. Por ejemplo, si tenemos la clase de gatos, podríamos clasificar aún más las cosas en las subclases "gato doméstico" y "gato salvaje", o incluso las subclases "gato moderno" y "gato prehistórico". Por otro lado, existen relaciones entre objetos, por ejemplo, si consideramos árboles y sus hojas, se podría representar que los objetos de árbol tienen colecciones de objetos hoja. Cada objeto hoja pertenece a un objeto árbol específico y no se pueden compartir entre ellos, y el ciclo de vida de la hoja está íntimamente unido a, y controlado por, el árbol. Este tipo de relación entre objetos se lo conoce como composición.

Sin embargo, si considera la relación entre computadora y periféricos, la relación de objeto es muy diferente. Un periférico, como un par de altavoces, se puede intercambiar entre varias computadoras; diferentes computadoras pueden incluso compartir los mismos periféricos. Igualmente, incluso si una computadora se tira, sus periféricos pueden sobrevivir y utilizarse en una nueva máquina. El ciclo de vida de los periféricos es normalmente independiente del ciclo de vida de la computadora. En UML, este tipo de relación de objeto se conoce como agregación.

Clases y objetos

La relación entre una clase y objetos de esa clase es una relación de clasificación (<<instantiate>>). El Manual de referencia de UML (Rumbaugh y colab., 2007) define una relación como una conexión entre elementos de modelo. Existen muchos tipos de relación en UML y exploraremos todos ellos.

La relación de clasificación (<<instantiate>>, o esInstanciaDe) entre objetos y clases se muestra en la Figura 5, no obstante, no es habitual representar en un mismo diagrama Clases e instancias. La flecha de puntos es una relación de dependencia a la que se le ha dado un significado especial por el estereotipo <<instantiate>>. Cualquier cosa dentro de << ... >> se conoce como un estereotipo, y los estereotipos son uno de los tres mecanismos de ampliación de UML. Un estereotipo es una forma de personalizar elementos de modelado, una forma de crear variaciones con nueva semántica. En este caso, el estereotipo <<instantiate>> convierte una dependencia ordinaria en una relación de instanciación entre una clase y los objetos de esa clase. Rumbaugh y colab. (2007) definen una dependencia como una relación entre dos elementos en los que un cambio en un elemento (el proveedor) puede afectar o proporcionar

información necesaria por el otro elemento (el cliente). Por la Figura 5, queda claro que la clase Cuenta debe ser el proveedor porque determina la estructura de todos los objetos de esa clase, y está claro que los objetos son clientes.

```
@startuml
class Cuenta {
    numeroDeCuenta: String
    propietario: String
    saldo: double
    retirar()
    depositar()
}
object "<u>cuentaDeNicolas:Cuenta</u>" as ctaNicolas {
    numeroDeCuenta = "12345"
    propietario = "Nicolás"
    saldo = 300,00
}
Cuenta <.. ctaNicolas : <<instantiate>>
object "<u>cuentaDeSol:Cuenta</u>" as ctaSol {
    numeroDeCuenta = "3456"
    propietario = "Sol"
    saldo = 1000,00
}
Cuenta <.. ctaSol : <<instantiate>>
object "<u>cuentaDeJulieta:Cuenta</u>" as ctaJulieta {
    numeroDeCuenta = "789"
    propietario = "Julieta"
    saldo = 250,00
}
Cuenta <.. ctaJulieta : <<instantiate>>
hide circle
@enduml
```

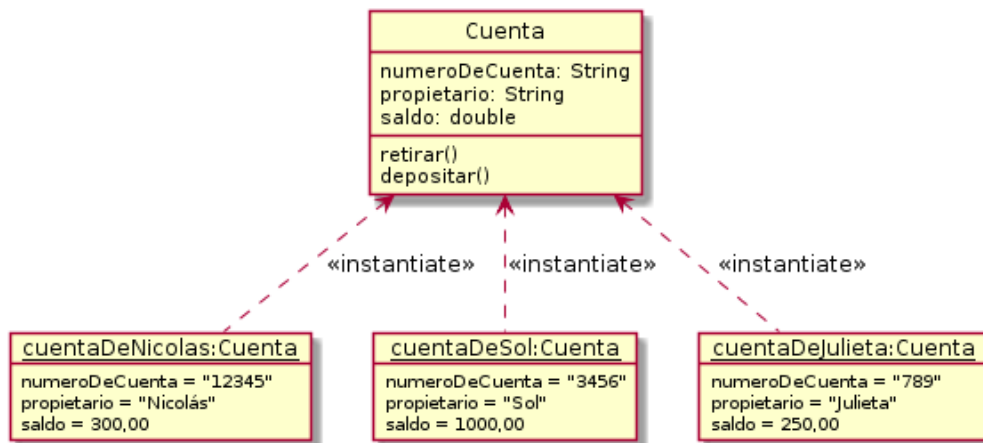


Figura 5. Instancias de Cuenta

Instanciar clases

Instanciar es la creación de nuevas instancias de elementos de modelo. En este caso, estamos instanciando objetos a partir de clases; estamos creando nuevas instancias de clases. UML trata de ser general y, por lo tanto, instanciar se aplica a otros elementos de modelado como clases y objetos. De hecho, instanciar captura la noción general de crear una instancia específica de algo a partir de una plantilla.

En la mayoría de los lenguajes de programación orientados a objetos, existen operaciones especiales, denominadas constructores, que realmente pertenecen a la propia clase en lugar de a los objetos de esa clase. Estas operaciones especiales se dice que tienen un ámbito de clase. La finalidad de las operaciones de constructor es crear nuevas instancias de la clase. El constructor destina memoria para el nuevo objeto, le asigna una identidad única y establece los valores iniciales de atributo para el objeto. También establece cualquier vínculo a otros objetos.

Notación de clase de UML

La sintaxis visual de UML para una clase es muy rica y para hacer que la sintaxis sea manejable es importante aplicar la noción UML de adornos opcionales. La única parte obligatoria de la sintaxis visual es el compartimiento de nombre con el nombre de clase en él. El resto de compartimientos y adornos son opcionales. Sin embargo, en la Figura 6 se incluyen todos los elementos posibles para representar una clase CuentaBancaria.

@startuml

```

class CuentaBancaria {
    -numeroDeCuenta: String
    -propietario: String
    -saldo: double = 0.0
    +{static} crear(elNumero: String, elPropietario:String)
    +depositar(cantidad: double)
    +retirar(cantidad: double)
}
  
```



```

+obtenerNumero(): String
+getPropietario(): String
+obtenerSaldo(): double
}
hide circle
@enduml

```

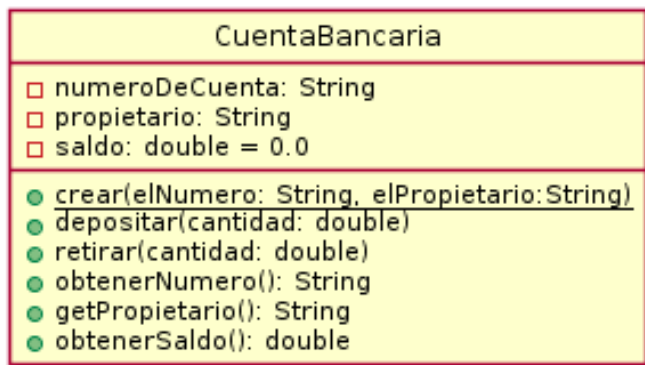


Figura 6. Clase CuentaBancaria

Los compartimientos y adornos que incluya en una clase en un diagrama de clase dependen por completo de la finalidad del diagrama. Si solamente está interesado en mostrar las relaciones entre varias clases, entonces puede estar contento con solamente el nombre del compartimiento. Si el diagrama es para ilustrar comportamiento de las clases, entonces probablemente añadirá el compartimiento de operación y las operaciones clave en cada clase. Por otro lado, el diagrama podría estar más "orientado a datos", quizá tratando de mostrar el mapeado de clases a tablas relacionales. En este caso, mostraría el compartimiento de nombre y el compartimiento de atributo, quizá mostrando también los tipos de atributos. Debería tratar de utilizar esta flexibilidad de UML para mostrar la cantidad correcta de información sobre diagramas de clase para que el punto en cuestión sea más claro y conciso.

En modelos de análisis, solamente necesita mostrar lo siguiente:

- Nombre de clase.
- Atributos clave.
- Operaciones clave.
- Estereotipos (si tienen trascendencia para el negocio).

Normalmente no muestra lo siguiente:

- Valores etiquetados.
- Parámetros de operación.
- Visibilidad.
- Valores de inicialización (a menos que tengan trascendencia para el negocio)

En los siguientes apartados examinaremos los compartimientos de nombre, atributo y operación en detalle.

Compartimiento de nombre

Aunque UML no impone ninguna convención de nombres para clases, existe una convención que se sigue de forma casi universal:

- El nombre de clase empieza con una letra mayúscula y luego en una combinación mayúscula-minúscula, con cada palabra empezando en mayúscula. Los símbolos especiales como signos de puntuación, guiones, subrayados y barras inclinadas se evitan siempre. Existe una buena razón para ello: estos símbolos se utilizan en lenguajes como HTML, XML y en sistemas operativos. Incluirlos en los nombres de clase o en los nombres de cualquier otro elemento de modelado puede llevar a consecuencias inesperadas cuando se genera código o documentación HTML/XML del modelo.
- Evite abreviaciones. Los nombres de clase deberían reflejar siempre los nombres de entidades del mundo real sin abreviaciones. Por ejemplo, CuentaBancaria (nombre elegido en Figura 6) siempre es preferible a CtaBancaria. La razón para ello es muy sencilla, las abreviaturas hacen que el modelo (y el código resultante) sea más difícil de leer. Cualquier tiempo que se ahorre escribiendo se pierde con el tiempo cuando se tiene que mantener el modelo o código abreviado.
- Si existen acrónimos específicos del dominio (por ejemplo, CRM) que se utilizan de forma común y entenderán todos los lectores del modelo, entonces podría utilizarlos. Sin embargo, recuerde que la forma larga podría ser preferible si hace que el modelo sea más claro.
- Las clases representan cosas, por lo que deberían tener un nombre que es un nombre común, en singular, por ejemplo, Persona, Dinero, CuentaBancaria.

Compartimiento de atributo

A continuación, se incluye la sintaxis de atributo UML:

visibilidad nombre: tipo[multiplicidad] = valorInicial

La única parte obligatoria de la sintaxis de atributo UML es el nombre de atributo, el resto de los elementos son opcionales. Los atributos se nombran en minúscula, si está compuesto por varias palabras, se coloca en mayúscula la primera letra de cada palabra, pero la primera letra de todas en minúsculas. Los nombres de atributo son normalmente nombres o frases nominales porque los atributos indican alguna cosa como un saldo de cuenta. Debe evitar símbolos especiales y abreviaturas.

En los siguientes apartados examinamos las partes opcionales de la sintaxis de atributo.

Visibilidad

El adorno de visibilidad (véase la Tabla 3) se aplica a atributos y operaciones dentro de la clase. También se puede aplicar a nombres de roles en relaciones. El análisis no satura los diagramas con visibilidad ya que ésta es una declaración del “cómo” en lugar de “qué”, por lo cual no emplearemos este constructor.

Los lenguajes de implementación pueden interpretar de forma diferente todos los tipos de visibilidad de UML, excepto pública. Esto es un punto importante, de hecho, los lenguajes pueden definir tipos adicionales de visibilidad que UML no soporta por defecto. Por lo general, esto no es un problema. Los lenguajes orientados a objetos más comunes, C++ y Java, e incluso

lenguajes semi-orientados a objetos como Visual Basic, no tienen problemas con la visibilidad pública, privada, protegida y de paquete, al menos como primera aproximación.

Tabla 3.

Adorno	Nombre de visibilidad	Semántica
+	Visibilidad pública	Cualquier elemento que puede acceder a la clase puede acceder a cualquiera de sus características con visibilidad pública.
-	Visibilidad privada	Solamente las operaciones dentro de la clase pueden acceder a características con visibilidad privada.
#	Visibilidad protegida	Sólo las operaciones dentro de la clase o dentro de hijos de la clase pueden acceder a características con visibilidad protegida.
~	Visibilidad de paquete	Cualquier elemento que esté en el mismo paquete que la clase, o en un subpaquete anidado, puede acceder a cualquiera de sus características con visibilidad de paquete.

La visibilidad depende del lenguaje de implementación y puede hacerse bastante compleja. El tipo exacto de visibilidad utilizada es una decisión detallada de implementación que normalmente tomaría el programador en lugar del analista/diseñador. Para un modelado general, las definiciones del estándar UML de público, privado, protegido y paquete son bastante adecuadas, sin embargo, cabe destacar que no coinciden con la visión de privado y público del modelo de objetos.

Tipo

El tipo de un atributo puede ser otra clase o un tipo primitivo. La especificación UML define cuatro tipos primitivos que se utilizan en la propia especificación UML y puede utilizarlos en sus modelos de análisis cuando quiera permanecer independiente de la plataforma. Estos tipos se listan en la Tabla 4.

Tabla 4.

	Tipo primitivo	Semántica
UML	Integer	Número entero.
	UnlimitedNatural	Número entero mayor o igual a cero. Infinito se lo representa como *.
	Boolean	Puede tomar los valores True (verdadero) o False (falso).
	String	Una secuencia de caracteres. Los valores van entre comillas, por ejemplo, "Nicolás".
OCL	Real	Número en coma flotante.

El Lenguaje de Restricción de Objetos (Object Constraint Language, OCL) es un lenguaje formal para expresar restricciones en modelos UML. OCL define operaciones estándar para los tipos UML primitivos y añade un nuevo tipo denominado Real.

Si su modelo se dirige a un lenguaje específico como Java o C#, entonces puede utilizar los tipos primitivos de ese lenguaje. Sin embargo, su modelo quedará unido a ese lenguaje.

Multiplicidad

La multiplicidad se utiliza ampliamente en diseño, pero también se puede utilizar en modelos de análisis ya que puede proporcionar una forma concisa de expresar ciertas restricciones de negocio relacionadas con el número de elementos que participan en una relación. La multiplicidad le permite modelar dos elementos totalmente diferentes al utilizar una expresión de multiplicidad sobre un atributo (véase la Figura 7).

```
@startuml
class Persona {
  -nombre: String[2..*]
  -direccion: String[3]
  -email: String[0..1]
}
hide circle
hide members
show fields
@enduml
```

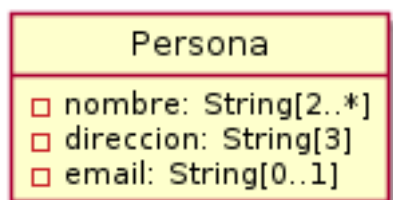


Figura 7. Especificación de la clase Persona con atributos nombre, direccion, email. nombre se componen de dos o más String. direccion se compone de tres String. email se compone de un String o nada (es opcional).

Colecciones: Si la expresión de multiplicidad resulta en un entero mayor que 1, entonces está especificando una colección del tipo. Por ejemplo, colores: Color [7] modelaría un atributo que es una colección de siete objetos Color que podría utilizar para modelar los colores del arco iris.

Valores nulos: Existe una diferencia en la mayoría de los lenguajes entre un atributo que contiene un objeto vacío como la cadena vacía y un atributo que apunta a ninguna parte, es decir, a la referencia del objeto null. Cuando un atributo referencia a null, esto significa que el objeto al que apunta todavía no se ha creado o ha dejado de existir. Algunas veces puede ser importante en el diseño detallado distinguir cuándo null es un valor posible para un atributo, y

puede modelar esto al utilizar la expresión especial de multiplicidad [0..1] . Tome el ejemplo email en la Figura 7: si el atributo email tiene el valor "" podría hacer que esto significara que ha pedido una dirección de correo electrónico y no se le ha dado. Por otro lado, si el atributo email apunta a null, puede hacer que esto signifique que todavía no ha pedido la dirección de correo electrónico, por lo que su valor es desconocido. Si utiliza distinciones como éstas, es esencial que grabe en la documentación de email la semántica precisa de "" vs. null. Como puede ver, ésta es una consideración de diseño bastante detallada, pero puede ser importante y de utilidad.

La Figura 7 le proporciona algunos ejemplos de sintaxis de multiplicidad.

Valor inicial

El valor inicial le permite especificar el valor que tomará un atributo cuando se instancia un objeto desde la clase. Esto se conoce como el valor inicial del atributo porque es el valor que el atributo toma en el momento de la creación del objeto. En diseño, es un buen estilo utilizar valores iniciales siempre que sea posible, ayuda a asegurarse de que los objetos de la clase siempre se crean en un estado válido y útil.

Los valores iniciales solamente se utilizan en análisis cuando pueden expresar o destacar una restricción importante de negocio. Esto no es muy habitual.

Sintaxis avanzada de atributo

Como cualquier otro elemento de modelado UML, puede ampliar la semántica de los atributos al prefijarlos con estereotipos. Siempre puede ampliar la especificación de un atributo al postfijar el atributo con valores etiquetados. Por ejemplo,

```
atributo{etiqueta 1= valor 1, etiqueta2=valor2, ...}
```

Puede almacenar cualquier información que desee en valores etiquetados. Por ejemplo, a veces se utiliza para almacenar información de versión como se muestra aquí:

```
dirección {añadidoPor= "Jim Arlow", fechaAñadido= "20/03/2004" }
```

En este ejemplo hemos grabado que Jim Arlow añadió el atributo dirección a una clase el 20 de marzo de 2004.

Compartimiento de operación

Las operaciones son funciones que están unidas a una clase en particular. Como tal, tienen todas las características de funciones:

- Nombre
- Lista de parámetros
- Tipo de retorno

La combinación del nombre de operación, tipos de todos los parámetros y el tipo de retorno es la firma (signatura) de operación, la sintaxis es la siguiente:

```
visibilidad nombre(dirección nombreParámetro: tipoParámetro= valorPredeterminado, ...):  
                                tipoRetorno
```

Toda operación de una clase debe tener una firma única ya que esta firma es la que asigna a la operación su identidad. Cuando un mensaje se envía a un objeto, la firma del mensaje se

compara con las firmas de operación definidas en la clase del objeto y si se encuentra coincidencia, la operación apropiada se invoca en el objeto.

Diferentes lenguajes de implementación tienen diferentes interpretaciones de lo que constituye una firma de operación. Por ejemplo, en C++ y Java, el tipo de retorno se ignora. Esto significa que dos operaciones en una clase que solamente difieren por el tipo de retorno, se considerarán la misma operación y generarán un error de compilador/intérprete. En Smalltalk, los parámetros y el tipo de retorno son todos del tipo Objeto, y por lo tanto solamente es el nombre de operación lo que constituye la firma.

Las operaciones se nombran poniendo en mayúscula la primera letra de cada palabra con la primera letra de todas en minúscula. Los nombres de operación son normalmente un verbo o frase verbal, y debe evitar símbolos especiales y abreviaturas. Todo parámetro de operación tiene la siguiente forma:

dirección nombreParámetro : tipoParámetro = valorPredeterminado

Una operación puede tener de cero a muchos parámetros. Los nombres de parámetro se realizan poniendo en mayúscula la primera letra de cada palabra con la primera letra de todas en minúscula y son normalmente un nombre o frase nominal ya que hacen referencia a algo que se pasa a la operación. Todo parámetro tiene un tipo que es una clase o tipo primitivo.

Dirección del parámetro

Los parámetros de operación pueden tener asignado una dirección:

operación (in p1: Integer, inout p2: Integer, out p3: Integer, return p4: Integer, return p5: Integer)

Si la dirección no se especifica, entonces por defecto es in. La semántica de la dirección del parámetro se resume en la Tabla 5.

Tabla 5.

Parámetro	Semántica
in p1: Integer	Predeterminado. La operación utiliza p1 como parámetro de entrada. El valor de p1 se utiliza por la operación de alguna forma. p1 no se cambia por la operación.
inout p2: Integer	La operación acepta p2 como parámetro de entrada/salida. El valor de p2 se utiliza por la operación de alguna forma y sirve para aceptar un valor como salida de la operación. p2 se puede cambiar por la operación.
out p3: Integer	La operación utiliza p3 como parámetro de salida. El parámetro sirve como un repositorio para aceptar un valor como

	salida de la operación.
return p4: Integer	La operación utiliza p4 como un parámetro de retorno. La operación devuelve p4 como uno de sus valores de retorno.

La semántica de in, inout y out son bastante sencillas, pero quizás return necesita algo más de explicación. Tendemos a esperar que las funciones devuelvan exactamente un objeto como se muestra a continuación:

valorMaximo = max (a , b)

valorMinimo = min (a, b)

donde a, b, valorMaximo y valorMinimo son Integer. De hecho, la sintaxis de operación normal soporta precisamente esta situación; toda operación puede tener un valor de retorno. Como ha visto, puede modelar estas operaciones normales en UML de la siguiente forma:

valorMaximo (a: Integer, b: Integer): Integer

Sin embargo, algunos lenguajes permiten que las operaciones devuelvan más de un valor. Por ejemplo, Python permite que las operaciones tengan múltiples valores de retorno. En Python podría escribir

valorMaximo, valorMinimo = maxMin (a, b)

donde maxMin (a , b) devuelve dos valores, el valor máximo y el valor mínimo. En UML, puede modelar esta operación de la siguiente forma:

maxMin (in a: Integer, in b: Integer, return maxValor: Integer, return minValor : Integer)

Puede ver que la dirección del parámetro return le permite modelar situaciones donde una operación tiene más de un valor de retorno.

Las direcciones de los parámetros son un aspecto de diseño y por lo tanto no tiene que preocuparse por ellas en el análisis. Las direcciones de los parámetros pueden ser muy importantes en diseño, especialmente si está generando código. Los generadores de código mapearán las direcciones de parámetros UML con el parámetro específico pasando semántica a su lenguaje destino.

Valores por defecto de parámetros

Puede asignar a un parámetro de operación un valor predeterminado. Cuando se invoca la operación, si no se ha asignado un valor para ese parámetro, se utiliza su valor por defecto. Los valores predeterminados son una cuestión de diseño y raramente los utiliza en análisis.

Sintaxis avanzada de operación

Puede extender la semántica de las operaciones al prefijarlas con estereotipos y postfijarlas con valores etiquetados.

<<estereotipo>> operación (...) {etiqueta1= valor1, etiqueta2= valor2, ...}

Operaciones de consulta

Toda operación tiene una propiedad denominada isQuery. Si establece esta propiedad en verdadero en su herramienta de modelado, la operación es una operación de consulta. Esto significa que no tiene efectos secundarios y no cambia el estado del objeto que se invoca. Una

operación que devuelve el valor de un atributo es una operación de consulta y debería tener `isQuery` establecido en `true`. El valor por defecto para `isQuery` es `false`.

Ámbito de aplicación

Hasta el momento ha visto que los objetos tienen sus propias copias de los atributos definidos en su clase por lo que diferentes objetos pueden tener diferentes valores de atributo. De forma similar, las operaciones que ha visto hasta el momento actúan sobre objetos específicos. Este es el caso normal, y decimos que estos atributos y operaciones tienen ámbito de instancia.

Sin embargo, algunas veces necesita definir atributos que tienen un solo valor compartido para cada objeto de la clase, y necesita operaciones (como operaciones de creación de objeto) que no operen sobre ninguna instancia de clase en particular. Decimos que estos atributos y operaciones tienen ámbito de clase. Las características de ámbito de clase proporcionan un conjunto de características globales para una clase entera de objetos.

Ámbito de instancia y ámbito de clase

La notación para atributos y operaciones de ámbito de instancia y ámbito de clase se muestra en la Figura 8. La semántica para los atributos y operaciones del ámbito de instancia y ámbito de clase se resume en la Tabla 6.

```
@startuml
class CuentaBancaria {
    -numeroDeCuenta: String
    - {static} cuenta: int = 0
    +{static} crear(elNumero: String)
    +obtenerNumeroDeCuenta(): String
    +{static} obtenerCuenta(): int
    +{static} incrementarCuenta()
    +{static} decrementarCuenta()
}
hide circle
@enduml
```

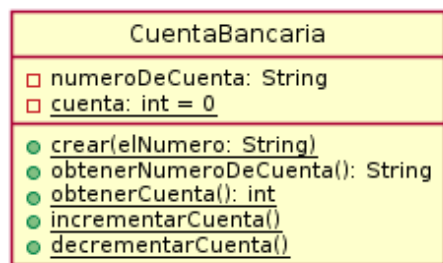


Figura 8. Notación para atributos y operaciones de ámbito de instancia y ámbito de clase. Ámbito de instancia: `numeroDeCuenta`, `obtenerNumeroDeCuenta()`. Ámbito de clase: `cuenta`, `crear()`, `obtenerCuenta()`, `incrementarCuenta()`, `decrementarCuenta()`.

Tabla 6

	Ámbito de instancia	Ámbito de clase
Atributos	Por defecto, los atributos tienen ámbito de instancia. Todo objeto de la clase tiene su propia copia de los atributos del ámbito de la instancia. Todo objeto puede tener diferentes valores de atributo de ámbito de instancia.	Los atributos se pueden definir como ámbito de clase Todo objeto de la clase comparte la misma copia de los atributos del ámbito de clase. Todo objeto tendrá los mismos valores de atributo de ámbito de clase.
Operaciones	Por defecto, las operaciones tienen ámbito de instancia. Toda invocación de una operación de ámbito de instancia se aplica a una instancia específica de la clase. No puede invocar una operación de ámbito de instancia a menos que tenga una instancia de clase disponible; esto significa que no puede utilizar una operación de ámbito de instancia de la clase para crear objetos de esa clase, ya que nunca crearía el primer objeto.	Las operaciones pueden definirse como ámbito de clase. La invocación de una operación del ámbito de clase no se aplica a ninguna instancia específica de la clase, en su lugar, puede pensar en operaciones de ámbito de clase como aplicadas a la propia clase. Puede invocar una operación de ámbito de clase incluso si no hay ninguna instancia de la clase disponible; esto es ideal para las operaciones de creación de objeto.

Siempre que una operación pueda acceder a otra característica de la clase está determinado por el ámbito de aplicación de la operación y por el ámbito de la característica que está tratando de acceder.

Las operaciones con ámbito de instancia pueden acceder a otros atributos y operaciones de ámbito de aplicación de instancia, y también a todos los atributos y operaciones del ámbito de aplicación de clase. Las operaciones del ámbito de operación de clase solamente pueden acceder a otras operaciones y atributos del ámbito de aplicación de clase. Las operaciones de ámbito de aplicación de clase no pueden acceder a operaciones de ámbito de aplicación de instancia porque:

- Puede que todavía no se haya creado ninguna instancia de clase.
- Incluso si existen instancias de clase, no sabe cuál utilizar.

Construcción y destrucción de objetos

Los constructores son operaciones especiales que crean nuevas instancias de clases; estas operaciones deben tener ámbito de aplicación de clase. Si tuvieran ámbito de aplicación de instancia, no habría forma de crear la primera instancia de una clase.

Los constructores son una consideración de diseño y por lo general no se muestran en los modelos de análisis.

Diferentes lenguajes tienen diferentes estándares para nombrar constructores. Un enfoque completamente genérico es llamar al constructor `create(...)`. Esto deja claro la intención de la operación. Sin embargo, Java, C# y C++, todos demandan que el nombre del constructor sea el mismo que el nombre de la clase.

Una clase puede tener muchos constructores, todos con el mismo nombre, pero cada uno distinguido por una lista de parámetros diferente. El constructor sin parámetros se conoce como el constructor predeterminado. Puede utilizar parámetros de constructor para inicializar valores de atributo en el punto de la construcción del objeto.

En modelos a nivel de análisis, por lo general no se preocupa de especificar los constructores (o destructores). Por lo general, no tiene ningún impacto o relación con la semántica de negocio de la clase. Si quiere mostrar operaciones del constructor, puede situar una operación `create()` sin parámetros como un marcador. De forma alternativa, puede especificar esos parámetros que son importantes desde una perspectiva de negocio.

Cuando llega a un diseño detallado, necesita especificar el nombre, tipos de parámetro y tipo de retorno de cada operación y esto incluirá especificación explícita de operaciones de constructor y destructor.

La destrucción de objetos no es tan sencilla como la construcción de objetos. Diferentes lenguajes orientados a objetos tienen diferente semántica para la destrucción de objetos.

Encontrar clases de análisis

Los resultados del workflow del proceso unificado Analizar un caso de uso son clases de análisis y realizaciones de caso de uso. Las realizaciones de caso de uso, son colaboraciones entre objetos que muestran cómo sistemas de objetos que interactúan pueden realizar el comportamiento del sistema expresado en los casos de uso.

Merece la pena mirar las entradas en el workflow Analizar un caso de uso:

- Modelo de negocio: Puede o no tener un modelo de negocio disponible que esté relacionado con el sistema que está modelando. Si lo tiene, éste es una excelente fuente de requisitos.
- Modelo de requisitos: Estos requisitos proporcionan entrada de utilidad para el proceso de modelado de caso de uso. En particular, los requisitos funcionales sugerirán casos de uso y actores. Los requisitos no funcionales sugerirán aquello que puede tener en mente cuando construya el modelo de caso de uso.
- Modelo de caso de uso.
- Descripción de arquitectura: Una instantánea de los aspectos importantes desde el punto de vista de la arquitectura del sistema. Puede incluir extractos de los modelos UML incorporados en un texto explicativo. Esto lo crean los arquitectos con entradas de analistas/diseñadores.

¿Qué son clases de análisis?

Las clases de análisis son clases que:

- Representan una abstracción en el dominio del problema.
- Deberían mapearse con conceptos de negocio del mundo real (y estar nombrados en consecuencia).

El dominio del problema es el dominio en el que surge primero la necesidad de un sistema de software (de ahí una actividad de desarrollo de software). Ésta es un área específica del negocio como venta online o CRM. Por último, todo el desarrollo de software comercial sirve alguna necesidad de negocio, sea ésa automatizar un proceso de negocio existente o desarrollar un nuevo producto que tenga un componente de software importante.

El aspecto más importante de una clase de análisis es que debería mapear de forma clara y nada ambigua con algún concepto de negocio del mundo real, como cliente, producto o cuenta. Sin embargo, esta declaración asume que los propios conceptos de negocio son claros y nada ambiguos y esto es raramente el caso. Por lo tanto, es el trabajo del analista orientado a objetos tratar de clarificar los conceptos de negocio inapropiados o confusos en algo que pueda formar la base de una clase de análisis. Esta es la razón por la que el análisis orientado a objetos puede ser difícil.

Por lo tanto, el primer paso para crear software orientado a objetos es aclarar el dominio del problema. Si contiene conceptos de negocio claramente definidos y tiene una estructura funcional sencilla, la solución está ahí para tomarla. La mayor parte de este trabajo se realizará en el workflow de requisitos en las actividades de captar requisitos y crear un modelo de caso de uso y glosario del proyecto. Sin embargo, mucha más aclaración ocurre en la construcción de clases de análisis y realizaciones de casos de uso.

Es importante que todas las clases en el modelo de análisis sean clases de análisis en lugar de clases que surgen de consideraciones de diseño (el dominio de solución). Cuando llega a un diseño detallado, puede encontrar que las clases de análisis se convierten en una o más clases de diseño.

Aunque anteriormente hablamos de objetos específicos, ahora entenderá que el verdadero objetivo del análisis orientado a objetos es encontrar las clases de esos objetos. De hecho, encontrar las clases correctas de análisis es la clave del análisis y diseño orientado a objetos. Si las clases no son correctas en análisis, entonces el resto del proceso de ingeniería de software, que se promulga en los workflows de requisitos y análisis, estará en peligro. Por lo tanto, es totalmente crucial que dedique tiempo al workflow de análisis para asegurarse de que el conjunto correcto de clases de análisis se ha identificado. Su tiempo estará bien empleado ya que con toda seguridad le ahorrará tiempo más adelante.

Anatomía de una clase de análisis

Las clases de análisis deberían presentar un conjunto de atributos de muy alto nivel. Estos indican los atributos que tendrán las clases de diseño resultantes. Podríamos decir que las clases de análisis capturan atributos candidatos para las clases de diseño. Las operaciones de clase de análisis especifican, en un alto nivel, los servicios clave que debe ofrecer la clase. En diseño, se convertirán en operaciones implementables. Sin embargo, una operación a nivel de análisis se desglosará en más de una operación a nivel de diseño.

Ya hemos tratado la sintaxis UML para clases en gran detalle, pero en análisis, solamente se utiliza un pequeño subconjunto de esa sintaxis. Por supuesto, el analista es libre de añadir cualquier adorno que considere necesario para hacer que el modelo sea más claro. Sin embargo, la sintaxis básica de una clase de análisis siempre evita los detalles de implementación. Después de todo, en análisis estamos tratando de capturar lo más importante o idea principal. Una forma mínima para una clase de análisis consta de lo siguiente:

- Nombre: Es obligatorio.
- Atributos: Los nombres de atributo son obligatorios, aunque solamente un subconjunto importante de atributos candidatos se pueden modelar en este punto. Los tipos de atributo se consideran opcionales.
- Operaciones: En análisis, las operaciones pueden ser declaraciones de responsabilidades de la clase a muy alto nivel. Los parámetros de operación y tipos de retorno solamente se muestran donde son importantes para entender el modelo.
- Visibilidad: Generalmente no se muestra.
- Estereotipos: Se pueden mostrar si mejoran el modelo.
- Valores etiquetados: Se pueden mostrar si mejoran el modelo.

Un ejemplo se proporciona en la Figura 9, donde se incluye la clase CuentaBancaria.

```
@startuml
class CuentaBancaria {
    numeroDeCuenta
    propietario
    saldo
    retirar()
    calcularIntereses()
    depositar()
}
hide circle
@enduml
```

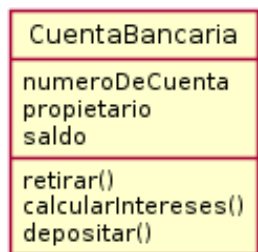


Figura 9.

La idea de una clase de análisis es que trata de capturar la esencia de la abstracción y deja los detalles de la implementación hasta que llega al diseño.

¿Qué hace una buena clase de análisis?

Podemos resumir lo que hace una buena clase de análisis en los siguientes puntos:

- Su nombre refleja su intención.
- Es una abstracción que modela un elemento específico del dominio del problema.
- Mapea en una característica claramente identificable del dominio del problema.
- Tiene un pequeño conjunto de responsabilidades bien definidas.
- Tiene una alta cohesión.
- Tiene un bajo acoplamiento.

En análisis, trata de modelar un aspecto del dominio del problema de forma precisa y concisa desde la perspectiva del sistema que trata de construir. Por ejemplo, si modela un cliente en un sistema bancario, querrá capturar el nombre, dirección del cliente, etc., pero no es probable que esté interesado por su preferencia de ventana o pasillo en un avión. Tiene que centrarse en aquellos aspectos del mundo real que son importantes desde la perspectiva del sistema que está creando.

Puede hacerse una primera idea de si una clase es una buena clase simplemente por su nombre. Si considera un sistema de comercio electrónico, Cliente parece que se referirá a algo bastante preciso en el mundo real y sería un buen candidato para una clase. CarroCompra también parecería una buena abstracción; sabemos casi de forma intuitiva cuál será su semántica. Sin embargo, algo como VisitanteSitioWeb parece tener una semántica bastante vaga y en realidad suena como un rol que un Cliente desempeña en relación con el sistema de comercio electrónico. Siempre debería tratar de buscar una abstracción concisa; algo que tiene una semántica clara y obvia.

Una responsabilidad es un contrato u obligación que una clase tiene con sus clientes. De forma esencial, una responsabilidad es un servicio que una clase ofrece a otras clases. Es crucial que sus clases de análisis tengan un conjunto cohesivo de responsabilidades que directamente estén en concordancia con el propósito de la clase (según se expresa por su nombre) y con el elemento del mundo real que la clase está modelando. Regresando al ejemplo CarroCompra, esperaríamos que esta clase tuviera responsabilidades como:

- Añadir artículo a carro.
- Eliminar artículo de carro.
- Mostrar artículos en carro.

Este es un conjunto cohesivo de responsabilidades, todas sobre mantener una colección de artículos que el cliente ha elegido. Es cohesivo porque todas las responsabilidades trabajan hacia el mismo objetivo, mantener el carro de compra del cliente. De hecho, podríamos resumir estas tres responsabilidades como una responsabilidad de muy alto nivel denominada mantener carro.

Ahora, podría también añadir las siguientes responsabilidades a CarroCompra:

- Validar tarjeta de crédito.
- Aceptar pago.
- Imprimir recibo.

Pero estas responsabilidades no parecen encajar con el propósito o semántica intuitiva de los carros de compra. No son tan cohesivas y se deberían asignar en cualquier otra parte, quizás a la clase EmpresaTarjetaCredito, una clase Facturar, y una clase ImprimirRecibo. Es importante

distribuir responsabilidades de forma apropiada sobre las clases de análisis para maximizar la cohesión dentro de cada clase. Por último, las buenas clases tienen la cantidad mínima de acoplamiento con otras clases. Medimos el acoplamiento entre clases por el número de otras clases con las que una clase dada tiene relaciones. Una distribución uniforme de responsabilidades entre clases tenderá a resultar en un acoplamiento bajo. La localización de control o de muchas responsabilidades en una sola clase tiende a incrementar el acoplamiento a esa clase.

Reglas generales de clases de análisis

Aquí tiene algunas reglas generales para crear clases de análisis bien estructuradas.

- De tres a cinco responsabilidades por clase: Normalmente las clases se deberían mantener lo más simples posibles, y esto normalmente limita el número de responsabilidades que pueden soportar a entre tres y cinco. Nuestro ejemplo anterior de CarroCompra es un buen ejemplo de una clase con un número pequeño y manejable de responsabilidades.
- Ninguna clase permanece sola: La esencia de buen análisis y diseño orientado a objetos es que las clases colaboran entre sí para proporcionar beneficio a los usuarios. Como tal, cada clase se debería asociar con un pequeño número de otras clases con las que colabore para distribuir el beneficio deseado. Las clases pueden delegar parte de sus responsabilidades a otras clases de ayuda que están dedicadas a esa función específica.
- Tenga cuidado de muchas clases muy pequeñas, algunas veces puede ser difícil encontrar el equilibrio correcto: Si el modelo parece tener muchas clases pequeñas con una o dos responsabilidades cada una, entonces debería examinar esto detenidamente con idea de consolidar algunas de las clases pequeñas en más grandes.
- Tenga cuidado de pocas clases, pero muy grandes: Lo contrario de lo anterior es un modelo que tiene pocas clases, donde muchas de ellas tienen un gran número (>5) de responsabilidades. La estrategia aquí es examinar estas clases en turno y ver si cada una se puede descomponer en dos o más clases más pequeñas con el número apropiado de responsabilidades.
- Tenga cuidado de los "functoids": Un "functoid" es una función procedimental normal disfrazada como una clase. Grady Booch contó la anécdota de un modelo de un sistema muy sencillo que tenía miles de clases. En un examen más detallado, toda clase tiene exactamente una operación denominada tonto(). Los functoid son siempre un peligro cuando los analistas acostumbrados a la técnica de la descomposición funcional de arriba abajo se acercan al análisis y diseño orientado a objetos la primera vez.
- Tenga cuidado de las clases omnipotentes: Estas son clases que parecen hacerlo todo. Busque clases con sistema o controlador en su nombre. La estrategia para tratar con este problema es ver si las responsabilidades de la clase omnipotente se encuentran en subconjuntos cohesivos. Si es así, quizá cada uno de estos conjuntos cohesivos de responsabilidades se pueda destacar en una clase aparte. Estas clases más pequeñas colaborarán para implementar el comportamiento ofrecido por la clase omnipotente original.

- Evite árboles de herencia muy profundos: La esencia de diseñar una buena jerarquía de herencia es que cada nivel de abstracción en la jerarquía debería tener una finalidad bien definida. Es fácil añadir muchos niveles que no sirven para ningún propósito de utilidad. De hecho, un error común es utilizar la herencia para implementar un tipo de descomposición funcional donde cada nivel de abstracción tiene solamente una responsabilidad. Esto conduce a un modelo complejo difícil de entender. En análisis, la herencia solamente se utiliza donde existe una jerarquía de herencia clara y obvia que surge directamente del dominio del problema.

En este último apartado tenemos que aclarar lo que queremos decir por árbol de herencia profundo. En análisis, donde las clases representan elementos de negocio, profundo serían tres niveles de herencia o más. Esto es porque los elementos de negocio tienden a formar jerarquías de herencia que son amplias en lugar de profundas. En diseño, donde el árbol consta de clases del dominio de solución, la definición de profundo depende del lenguaje de implementación al que se dirija, En Java, C++, C#, Python y Visual Basic, consideramos tres o más niveles como profundo. Sin embargo, en Smalltalk los árboles de herencia pueden ser mucho más profundos que esto debido a la estructura del sistema Smalltalk.

Encontrar clases

Meyer en Object Oriented Software Construction (Meyer, 1997) indica que no existe un algoritmo sencillo para encontrar las clases de análisis correctas. Si un algoritmo así existe, entonces equivaldrá a un medio infalible de diseñar software orientado a objeto y esto es tan improbable como encontrar un medio infalible de probar los teoremas matemáticos.

Aun así, existen técnicas probadas y testadas que llevan hacia una buena respuesta y las presentamos aquí. Estas implican analizar texto y entrevistar a usuarios y expertos de dominio. Pero a pesar de todas las técnicas, encontrar las clases correctas" depende de la perspectiva, habilidad y experiencia del analista.

Encontrar clases al utilizar análisis sustantivo/verbo

El análisis sustantivo/verbo es una forma muy sencilla de analizar texto para tratar de encontrar clases, atributos y responsabilidades. En esencia, los sustantivos y frases nominales en el texto indican clases o atributos de clases, y los verbos y frases verbales indican responsabilidades u operaciones de una clase. El análisis sustantivo/verbo se lleva utilizando muchos años y funciona bien ya que está basado en un análisis directo del lenguaje del dominio del problema. Sin embargo, tiene que estar pendiente de sinónimos y homónimos ya que esto puede dar lugar a clases falsas.

También debería tener mucho cuidado si el dominio del problema está mal definido. En este caso, trate de recopilar tanta información del dominio de tantas personas como le sea posible. Busque dominios de problemas similares fuera de su organización.

Quizás el aspecto más difícil del análisis sustantivo/verbo es encontrar las clases ocultas. Estas son clases que son intrínsecas al dominio del problema, pero puede que nunca se mencionen explícitamente. Por ejemplo, en un sistema de reservas para una empresa de vacaciones, oír a los grupos de decisión hablar sobre reservas, etc., pero la abstracción más importante, pedido, es posible que nunca se mencione explícitamente si no existe en los sistemas de negocio actuales. Normalmente sabe cuándo ha encontrado una clase oculta porque todo el modelo parece encajar de repente con la introducción de esta nueva abstracción. Esto sucede

sorprendentemente a menudo, de hecho, si alguna vez tenemos problemas con un modelo de análisis y no parece que tenga sentido, vamos a buscar clases ocultas. Esto nos hace formularnos preguntas y mejorar nuestro entendimiento del dominio del problema.

Procedimiento de análisis sustantivo/verbo

El primer paso en el análisis sustantivo/verbo es recopilar tanta información importante como sea posible. Algunas fuentes posibles de información son:

- El modelo de requisitos.
- El modelo de caso de uso.
- El glosario del proyecto.
- Cualquier otra cosa (arquitectura, documentos de visión, etc.).

Después de recopilar la documentación, analízela de forma sencilla al destacar (o grabar de alguna forma) lo siguiente:

- Sustantivos, por ejemplo, vuelo.
- Frases nominales, por ejemplo, número de vuelo.
- Verbos, por ejemplo, asignar.
- Frases verbales, por ejemplo, verificar tarjeta crédito.

Los sustantivos y frases nominales pueden indicar clases o atributos de clase. Los verbos y frases verbales pueden indicar responsabilidades de clases.

Si se encuentra con algún término que no entiende durante este proceso, busque una aclaración inmediata en un experto y añada el término al glosario del proyecto. Tome la lista de sustantivos, frases nominales, verbos y frases verbales y utilice el glosario del proyecto para solucionar cualquier sinónimo y homónimo. Esto crea una lista de clases, atributos y responsabilidades candidatos.

Una vez que tiene esta lista hace una asignación tentativa de los atributos y responsabilidades a las clases. Puede realizar esto al incorporar las clases en herramienta de modelado y añadir las responsabilidades como operaciones a las clases. Si ha encontrado algún atributo candidato, puede asignarlo también a las clases. También podría haberse hecho una idea de las relaciones entre ciertas clases (los casos de uso son una buena fuente de estos), por lo que puede añadir algunas asociaciones candidatas. Esto le proporciona un primer modelo de clase que puede mejorar con más análisis.

Encontrar clases al utilizar análisis CRC

Una buena forma de hacer que el usuario se implique en la búsqueda de clases es utilizar el análisis CRC, CRC significa Clase, Responsabilidades y Colaboradores. Esta técnica utiliza post-it como herramienta.

Empieza marcando algunos post-it como muestra la Figura 10. La nota se divide en tres compartimentos. En el compartimiento superior graba el nombre de la clase candidata; en el compartimiento izquierdo, las responsabilidades; y en el derecho los colaboradores. Los colaboradores son otras clases que pueden colaborar con esta clase para realizar una parte de la funcionalidad del sistema. El compartimiento de colaboradores proporciona una forma de grabar relaciones entre clases. Otra forma de capturar relaciones (que preferimos) es pegar las notas en una pizarra y dibujar líneas entre las clases colaboradoras.

Nombre de clase: CuentaBancaria	
Responsabilidades: Mantener saldo	Colaboradores: Banco

Figura 10.

Procedimiento de análisis CRC

El análisis CRC siempre se debería utilizar junto con análisis sustantivo/verbo de casos de uso, requisitos, glosario y otra documentación relevante, a menos que el sistema sea muy sencillo. El procedimiento de análisis CRC es sencillo y la clave está en separar la información que se recopila del análisis de información. CRC se ejecuta mejor como una actividad de dos fases.

Fase 1. Tormenta de ideas: recopilar información

Los participantes son analistas, grupos de decisión y expertos de dominio orientados a objeto. También necesita un facilitador. El procedimiento es el siguiente:

1. Explicar que se trata de una verdadera tormenta de ideas.
 1. Todas las ideas se aceptan como buenas ideas.
 2. Las ideas se graban, pero no se debaten, nunca se discute sobre algo, simplemente se escribe y se sigue adelante. Todo se analizará más adelante.
2. Pedir a los miembros del equipo que nombren los elementos que funcionan en su dominio de negocio, por ejemplo, cliente, producto.
 1. Escribirlo en un post-it; se trata de una clase candidata o atributo de una clase.
 2. Pegar la nota en una pared o pizarra.
3. Pedir al equipo que indique las responsabilidades que pudieran tener; grabarlas en el compartimiento de responsabilidades de la nota.
4. Trabajar con el equipo, tratar de identificar clases que podrían trabajar conjuntamente. Reorganizar las notas en la pizarra para reflejar esta organización y dibujar líneas entre ellas. De forma alternativa, grabar a los colaboradores en el compartimiento de colaboradores de la nota.

Fase 2. Analizar información

Los participantes son analistas y expertos de dominio orientados a objetos. ¿Cómo decide qué post-it deberían convertirse en clases y cuáles deberían convertirse en atributos? Como se ha indicado anteriormente, las clases de análisis deben representar una abstracción concisa en el dominio del problema. Ciertos post-it representarán conceptos clave de negocio y claramente necesitan convertirse en clases. Otras notas pueden convertirse en clases o atributos. Si una nota parece ser una parte de otra nota, ésta es una buena indicación de que representa un atributo. Igualmente, si una nota no parece ser particularmente importante o tiene un comportamiento poco interesante, vea si se puede hacer un atributo de otra clase.

Si se tiene dudas sobre una nota, hágala una clase. Lo importante es acertar y luego cerrar este proceso; siempre puede modificar el modelo más adelante.

Encontrar clases al utilizar los estereotipos RUP

Una técnica de utilidad procede de RUP en la forma de estereotipos RUP. La idea es que considera tres tipos distintos de clases de análisis durante su actividad de análisis. Esta es una forma de centrar su análisis en aspectos específicos del sistema. Consideramos esto una técnica opcional que puede utilizar para complementar las técnicas de análisis importantes, sustantivo/verbo y CRC presentadas anteriormente.

Tres tipos distintos de clase de análisis se pueden distinguir por los estereotipos que se muestran en la Tabla 7.

Tabla 7.

Tipo de clase (estereotipo)	Semántica
interfaz	Una clase mediadora entre el sistema y su entorno.
control	Una clase que encapsula comportamiento específico de caso de uso.
entidad	Una clase que se utiliza para modelar información persistente sobre algo.

Encontrar clases de interfaz

Estas clases existen en el límite de su sistema y se comunican con actores externos. Encuentra esas clases al considerar el sujeto (límite del sistema) y descubrir qué clases median entre el sujeto y su entorno. Según RUP existen tres tipos de clase:

- Clases de interfaz de usuario: Clases que hacen de interfaz entre el sistema y los usuarios.
- Clases de interfaz del sistema: Clases que hacen de interfaz con otros sistemas.
- Clases de interfaz de dispositivo: Clases que hacen de interfaz con dispositivos externos como sensores.

Toda comunicación entre un actor y un caso de uso en su modelo debe estar activada por algún objeto en su sistema. Estos objetos son instancias de clases de interfaz. Puede averiguar qué tipo de clase límite está indicado al considerar lo que representa el actor. Si representa una persona, entonces la clase será de interfaz de usuario. Si representa un sistema, la clase será de interfaz de sistema; y si representa un dispositivo, la clase se corresponderá con una de interfaz de dispositivo.

Cuando una clase interfaz sirve más de un actor, estos actores deberían ser del mismo tipo (representando una persona, sistema o dispositivo). Puede ser una indicación de mal diseño si una clase límite sirve actores de diferentes tipos.

Puesto que todavía se encuentra en análisis, es importante mantener estas clases en el nivel correcto de abstracción. Por ejemplo, cuando se modela una clase interfaz que representa una GUI, simplemente modele la ventana de nivel superior y deje todos los detalles que componen la ventana al diseño. De forma alternativa, puede introducir una clase tonta que representa toda la interfaz de usuario.

De forma similar, con clases de interfaz de sistema y clases de interfaz de dispositivo, está preocupado con capturar el hecho de que existe una clase que media entre su sistema y alguna otra cosa, pero no con los detalles específicos de esa clase. Decidirá estos detalles más adelante en el diseño.

Por ejemplo, si escribe un sistema de comercio electrónico que necesita actuar de interfaz con un sistema de inventario, puede representar la interfaz al sistema de inventario por medio de una clase denominada Interfaz Inventario. Esto es suficiente detalle para un modelo de análisis.

Encontrar clases de control

Estas clases son controladores, sus instancias coordinan comportamiento del sistema que corresponde a uno o más casos de uso. Encuentra clases de control al considerar el comportamiento del sistema como se describe por los casos de uso y averiguando cómo ese comportamiento se debería dividir entre las clases de análisis. El comportamiento sencillo se puede distribuir entre clases interfaz o entidad, pero el comportamiento más complejo, como el procesamiento de pedidos, está mejor localizado al introducir una clase controlador adecuada como GestorPedido. Algunos moderadores a menudo indican las clases de control al anexar Gestor o Controlador al nombre de la clase.

El punto clave cuando se trabaja con clases de control es dejar que las clases surjan de forma natural del propio dominio del problema. Algunos moderadores introducen de forma artificial una clase de control para cada caso de uso para controlar o ejecutar ese caso de uso. Esta es una aproximación peligrosa que lleva a un modelo que se parece más a una descomposición funcional de arriba a abajo que a un verdadero modelo de análisis orientado a objetos.

En el mundo real, los controladores que surgen directamente del dominio del problema (en lugar de como una consecuencia de una técnica de análisis específica) tienden a participar en varios casos de uso. Un buen ejemplo podría ser un controlador como una clase Secretario que está implicada en muchos de los casos de uso que describen un sistema de registro de cursos. De forma similar, un solo caso de uso puede requerir la participación de muchas clases de control.

Si encuentra que una clase controlador tiene un comportamiento muy complicado, esto indica que puede desglosarlo en dos o más controladores más sencillos que implementa cada uno un subconjunto cohesivo de ese comportamiento. Cada una de las clases más sencillas que identifique puede ser algo que ocurre de forma natural en el dominio del problema. Por ejemplo, cuando diseña un sistema de registro de cursos, podría introducir una clase de control denominada ControladorRegistroCurso que coordina todo el proceso. Pero dicha clase tiene un comportamiento complejo, por lo que podría decidir descomponerla en conjunto de clases de colaboración gestionando cada una de ellas uno o dos aspectos de ese comportamiento. ControladorRegistroCurso se podría descomponer en las clases Secretario, GestorCurso y Gestor Personal. Observe que cada una de estas clases representa algo que existe en el dominio del problema.

Una buena forma de explorar clases controlador es imaginarse en el rol de la clase. ¿Qué haría en esa situación?

Encontrar clases de entidad

Estas clases modelan información sobre algo y normalmente tienen un comportamiento muy sencillo que equivale a poco más que a obtener y establecer valores. Las clases que representan información persistente como las direcciones (una clase Dirección) y personas (una clase Persona) son clases de entidad. Las clases de entidad:

- Participan en muchos casos de uso.
- Están manipuladas por clases de control.
- Proporcionan información y aceptan información de clases de interfaz.
- Representan elementos clave gestionados por el sistema (por ejemplo, Cliente).
- A menudo son persistentes.

Las clases de entidad expresan la estructura lógica de datos del sistema. Si tiene un modelo de datos, entonces las clases de entidad están íntimamente relacionadas con entidades o tablas en este modelo.

Encontrar clases de otras fuentes

Junto con el análisis sustantivo/verbo, análisis CRC y estereotipos RUP, merece la pena recordar que existen muchas otras fuentes potenciales de clases que se deberían considerar. Igual que buscar abstracciones concisas que mapeen con elementos del mundo real en el dominio del problema, obviamente puede buscar clases en el mundo real.

- Los objetos físicos como avión, gente y hoteles pueden indicar clases.
- Los documentos son otra fuente estupenda de clases. Los recibos y pedidos, pueden indicar clases posibles. Sin embargo, debe tener mucho cuidado cuando examina todos estos papeles. En muchas empresas éste ha evolucionado con los años para soportar exactamente los procesos redundantes de negocio que el nuevo sistema podría tratar de reemplazar. Lo último que quiere hacer como analista/diseñador orientado a objetos es automatizar sistemas obsoletos basados en papel.
- Las interfaces conocidas con el mundo exterior, como pantallas, teclados, periféricos y otros sistemas, también pueden ser una fuente de clases candidatas, especialmente para sistemas embebidos.
- Entidades conceptuales son elementos que son cruciales para la operación del negocio, pero no se manifiestan como elementos concretos. Como ejemplo de esto, podría ser un ProgramaFidelidad. Claramente, el propio programa no es un elemento concreto, pero sigue siendo una abstracción cohesiva y por lo tanto se puede justificar su modelado como una clase.

Patrones arquetipo

Arlow y colab. (2004), en el libro Enterprise Patterns and MDA, describen un conjunto de patrones de conceptos de negocio. Como tales, se pueden modelar una vez y luego reutilizar, en lugar de modelarse una y otra vez en cada sistema de negocio. La idea del libro es que puede utilizar estos patrones como tales o modificarlos para construir su modelo de análisis a partir de componentes de modelo.

Los autores proporcionan los siguientes patrones:

- CRM
- Inventario Dinero

- Pedido
- Tercero
- Relación con terceros
- Producto
- Cantidad
- Regla

Cada uno de estos patrones está muy detallado. Si puede reutilizar uno de estos patrones puede ahorrarse muchos hombres/día incluso hombres/meses de trabajo. Incluso si el patrón no es del todo apropiado para lo que está tratando de modelar, puede proporcionarle ideas de utilidad para sus propias clases de análisis en lugar de partir de una página en blanco. Esta es probablemente la forma más eficaz de encontrar clases para sus modelos.

Un enfoque similar es el que presenta Fowler en <https://martinfowler.com/tags/analysis%20patterns.html>.

Crear un modelo de análisis de primer nivel

Para crear un modelo de análisis de primer nivel necesita consolidar las salidas del análisis sustantivo/verbo, análisis CRC, estereotipos RUP y una consideración de otras fuentes de clases (especialmente patrones) en un solo modelo UML en una herramienta de modelado. Realice la consolidación de la siguiente manera.

- Compare todas las fuentes de clases.
- Consolide las clases de análisis, atributos y responsabilidades de las diferentes fuentes e incorpórelas en una herramienta de modelado.
 - Utilice el glosario del proyecto para resolver sinónimos y homónimos.
 - Busque diferencias en los resultados de las tres técnicas. Las diferencias indican áreas donde se podría hacer más trabajo. Resuelva estas diferencias ahora o destáquelas para más adelante.
- Los colaboradores (o líneas entre los post-it en la pizarra) representan relaciones entre clases.
- Mejore el nombrado de clases, atributos y responsabilidades para seguir cualquier convención estándar de nombres que su empresa tenga.

El resultado de esta actividad es un conjunto de clases de análisis donde cada clase puede tener algunos atributos clave y debería tener entre tres a cinco responsabilidades. Este es su primer modelo de análisis de primer nivel.

Relaciones

Las relaciones son conexiones semánticas (significativas) entre elementos de modelado; son la forma de UML de conectar elementos juntos. Ya ha visto algunos tipos de relaciones:

- Entre actores y casos de uso (asociación).
- Entre casos de uso y casos de uso (generalización, <<include>> , <<extend>>).
- Entre actores y actores (generalización).

En esta sección exploramos conexiones entre objetos y conexiones entre clases. Empezamos con vínculos y asociaciones y, luego, examinamos la generalización y la herencia.

Para crear un sistema orientado a objetos que funcione, no puede dejar que los objetos permanezcan solos, aislados. Necesita conectarlos para que puedan realizar un trabajo de

utilidad para beneficiar a los usuarios del sistema. Las conexiones entre objetos se denominan vínculos, y cuando los objetos funcionan conjuntamente se dicen que colaboran.

Si existe un vínculo entre dos objetos, debe existir alguna conexión semántica entre sus clases. Esto es sentido común; para que los objetos se comuniquen directamente entre sí, las clases de esos objetos deben saber del resto de alguna forma. Las conexiones entre clases se conocen como asociaciones. Los vínculos entre objetos son instancias de las asociaciones entre sus clases.

¿Qué es un vínculo?

Para crear un programa orientado a objetos, los objetos se tienen que comunicar entre sí. De hecho, un programa orientado a objetos ejecutable es una comunidad de objetos que cooperan.

Un vínculo es una conexión semántica entre dos objetos que permite enviar mensajes de un objeto a otro. Un sistema orientado a objetos ejecutable contiene muchos objetos que van y vienen y muchos vínculos (que también van y vienen) que unen esos objetos. Los mensajes se pasan entre los objetos. Al recibir un mensaje, un objeto invocará su operación correspondiente.

Los vínculos se implementan de formas diferentes por diferentes lenguajes orientados a objetos. Java implementa los vínculos como referencias de objeto; C++ puede implementar los vínculos como referencias o por una inclusión directa de un objeto por otro. Cualquiera que sea el enfoque, un requisito mínimo para un vínculo es que al menos uno de los objetos tiene que tener una referencia de objeto al otro.

Diagramas de objeto

Un diagrama de objeto es un diagrama que muestra objetos y sus relaciones en un punto en el tiempo. Es como una foto de parte de un sistema ejecutable orientado a objetos en un instante determinado, mostrando los objetos y vínculos entre ellos.

Los objetos que están conectados por vínculos pueden adoptar varios roles relativos entre ellos. En la Figura 11 puede ver que el objeto lucas adopta el rol de director en su vínculo con el objeto clubLibro. Indica esto en el diagrama del objeto al situar el nombre del rol en el extremo apropiado del vínculo. Puede situar nombres de rol en uno o ambos extremos de un vínculo. En este caso, el objeto clubLibro siempre desempeña el rol de club por lo que no hay necesidad de mostrarlo en el diagrama; no añadirá nada a nuestro entendimiento de las relaciones de objetos.

@startuml

object "<u>clubLibro:Club</u>" as clubLibro

object "<u>lucas:Persona</u>" as lucas

object "<u>matias:Persona</u>" as matias

object "<u>clara:Persona</u>" as clara

clubLibro -- "director" lucas

clubLibro -- "socio" matias

clubLibro -- "secretaria" clara

@enduml

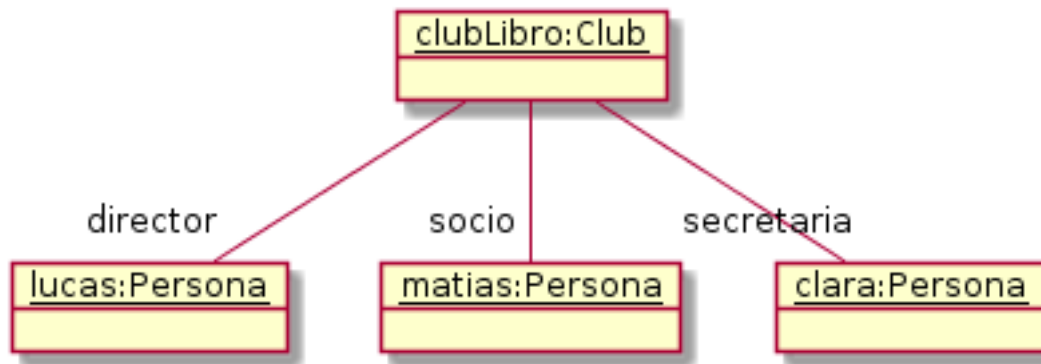


Figura 11.

La Figura 11 nos dice que en un punto determinado en el tiempo, el objeto lucas desempeña el rol de director. Sin embargo, es importante darse cuenta que los vínculos son conexiones dinámicas entre objetos. Es decir, no están necesariamente fijas en el tiempo. En este ejemplo, el rol director puede pasar en algún momento a matias o clara y podríamos crear un diagrama de objeto para mostrar este nuevo estado. Normalmente, un solo vínculo conecta exactamente dos objetos, como muestra en la Figura 11. Sin embargo, UML permite que un solo vínculo conecte más de dos objetos. Esto se conoce como un vínculo n-ario y se muestra con ruta a cada objeto participante. Muchos modeladores consideran esto innecesario. Raramente se utiliza y las herramientas de modelado UML no siempre lo soportan, por lo que no decimos nada más de esto aquí. En la Figura 11 en más detalle, puede ver que existen tres vínculos entre cuatro objetos:

- Un vínculo entre clubLibro y lucas.
- Un vínculo entre clubLibro y matias.
- Un vínculo entre clubLibro y clara.

En la Figura 11 los vínculos son bidireccionales, por lo tanto puede decir correctamente que el vínculo conecta lucas a clubLibro o que el vínculo conecta clubLibro a lucas. Si un vínculo es unidireccional, utiliza la navegabilidad para especificar en qué dirección los mensajes pueden pasar por el vínculo.

Puede mostrar la navegabilidad al situar una punta de flecha (navegable) o (no navegable) en el extremo de un vínculo. Los mensajes solamente pueden fluir en la dirección indicada por la punta de flecha. La especificación UML 2 permite formas diferentes de modelado para mostrar la navegabilidad, sin embargo utilizamos la siguiente especificación:

- Las asociaciones bidireccionales no tienen flechas.
- Las asociaciones unidireccionales tienen una sola flecha.

Por ejemplo, la Figura 12 muestra que el vínculo entre : Persona y :Direccion es unidireccional. Esto significa que el objeto :Persona tiene una referencia de objeto con el objeto :Direccion, pero no viceversa. Los mensajes solamente se pueden enviar de :Persona a :Direccion.

@startuml

object "<u>:Persona</u>" as unaPersona

object "<u>:Direccion</u>" as unaDireccion

unaPersona --> unaDireccion

@enduml

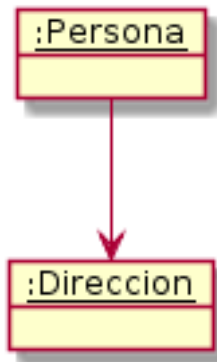


Figura 12.

Asociación

Las asociaciones son relaciones entre clases. Igual que los vínculos conectan objetos, las asociaciones conectan clases. El punto importante es que para que exista un vínculo entre dos objetos, debe haber una asociación entre las clases de esos objetos. Esto es porque un vínculo es una instancia de una asociación, igual que un objeto es una instancia de una clase. La Figura 13 muestra la relación entre clases y objetos, y entre vínculos y asociaciones. Puesto que no puede tener un vínculo sin una asociación, está claro que los vínculos dependen de las asociaciones. Para hacer explícita la semántica de la dependencia entre asociaciones y vínculos, se indicó en el nombre del vínculo el nombre de la asociación precedido por `(:ParticipaEn)`.

@startuml

class Club

class Persona

Club -- Persona : ParticipaEn

object "<u>matias:Persona</u>" as matias

object "<u>clubLibro:Club</u>" as clubLibro

Persona <.. matias : <<instantiate>>

Club <.. clubLibro : <<instantiate>>

clubLibro -- "socio" matias : ":ParticipaEn"

@enduml

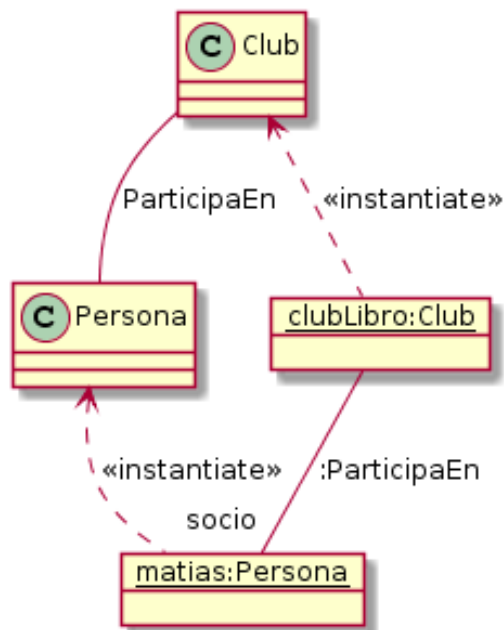


Figura 13.

La semántica de la asociación básica es muy sencilla, una asociación entre clases indica que puede tener vínculos entre objetos de esas clases. Existen otras formas de asociación (agregación y composición) que examinaremos luego.

Las asociaciones pueden tener:

- Un nombre de asociación.
- Nombres de roles.
- Multiplicidad.
- Navegabilidad.

Los nombres de asociación deberían ser frases verbales porque indican una acción que el objeto fuente está realizando sobre el objeto destino. El nombre también se podría prefijar o postfixar con una pequeña punta de flecha en negro para indicar la dirección en la que se debería leer el nombre de asociación. Los nombres de asociación se escriben poniendo en mayúscula la primera letra de cada palabra con la primera letra de todas en minúscula.

En el ejemplo de la Figura 14 se lee la asociación de la siguiente manera: una Empresa emplea muchas Personas. Aunque la flecha indica la dirección en la que se debería leer la asociación, siempre puede leer las asociaciones también en la otra dirección. Por lo tanto, en la Figura 14 puede decir: cada Persona está empleada exactamente por una Empresa en cualquier momento en el tiempo.

```

@startuml
class Empresa
class Persona
Empresa "1" -- "*" Persona : Emplea >
hide circle
@enduml

```

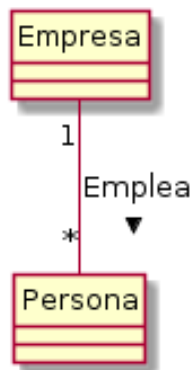


Figura 14.

De forma alternativa, puede asignar nombres de rol a las clases en uno o ambos extremos de la asociación. Estos nombres de roles indican los roles que los objetos de esas clases desempeñan cuando están vinculados por instancias de esta asociación. En la Figura 15 puede ver que un objeto Empresa desempeñará el rol empleador y objetos Persona desempeñará el rol empleado cuando estén vinculados por instancias de esta asociación. Los nombres de roles deberían ser nombres o frases nominales ya que nombran un rol que pueden desempeñar objetos.

```

@startuml
class Empresa
class Persona
Empresa "empleador 1" -- "empleado *" Persona : Emplea >
hide circle
@enduml
  
```

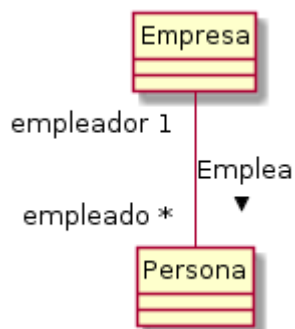


Figura 15.

Las asociaciones pueden tener un nombre de asociación o nombres de rol. Situar ambos nombres de rol y nombres de asociación en la misma asociación es teóricamente legal, pero no es habitual. La clave para buenos nombre de asociación y nombres de roles es que se deberían leerse bien.

En la Figura 14 una Empresa emplea muchas Personas, se lee muy bien. Leer la asociación al contrario puede decir que una Persona está empleada exactamente por una Empresa en

cualquier momento en el tiempo; se sigue también leyendo bien. De forma similar, los nombres de rol en la Figura 15 indican claramente los roles que los objetos de estas clases desempeñarán cuando se vinculan de esta forma en particular.

Multiplicidad

Las restricciones son uno de los tres mecanismos de extensibilidad de UML y multiplicidad es uno de estos tipos de restricción, quizás el más común. La multiplicidad restringe el número de objetos de clase que se pueden implicar en una relación determinada en cualquier momento en el tiempo. La frase "en cualquier momento en el tiempo" es vital para entender las multiplicidades. Considerando la Figura 15, puede ver que en cualquier momento en el tiempo un objeto Persona está empleado por exactamente un objeto Empresa (Cada Persona es empleado de una Empresa). Sin embargo, con el tiempo un objeto Persona podría estar empleado por una serie de objetos Empresa (en diferentes instantes de tiempo, pero siempre va a estar empleado por una única Empresa).

Examinando la Figura 15, puede ver algo más que es interesante. Un objeto Persona nunca puede estar desempleado, siempre está empleado por exactamente un objeto Empresa. La restricción por lo tanto incorpora dos reglas de negocio en este modelo:

- Que los objetos Persona solamente se pueden emplear por una Empresa, cada vez.
- Que los objetos Persona siempre deben estar empleados.

Que éstas sean restricciones razonables o no depende por completo de los requisitos del sistema que esté modelando, pero esto es lo que dice el modelo. Puede ver que las limitaciones de multiplicidad son muy importantes; pueden codificar reglas clave de negocio en su modelo. La multiplicidad se especifica como una lista de intervalos separados por coma, donde cada intervalo es de la forma:

mínimo .. máximo

mínimo y máximo pueden ser enteros o cualquier expresión que tenga un resultado entero.

Si la multiplicidad no se indica explícitamente, está pendiente, no existe multiplicidad predeterminada en UML. De hecho, es un error común de modelado UML asumir que una multiplicidad sin decidir por defecto indica una multiplicidad de 1. Algunos ejemplos de sintaxis de multiplicidad se indican en la Tabla 8.

Tabla 8.

Multiplicidad	Semántica
0..1	Cero o 1
1	Exactamente 1 (es equivalente a 1..1)
*	Cero o más (es equivalente a 0..*)
1..*	1 o más
1..6	1 a 6
1..3, 7..10, 15, 19..*	1 a 3, o 7 a 10, o 15, o 19 a muchos

El ejemplo en la Figura 16 se ilustra que la multiplicidad es una potente restricción que a veces tiene un gran efecto sobre lo que se está representando. Si lee el ejemplo detenidamente ve que:

- Una Empresa puede tener exactamente siete empleados.
- Una Persona puede estar empleada por exactamente una Empresa (en este modelo una Persona no puede tener más de un trabajo a la vez).
- Una CuentaBancaria puede tener exactamente un propietario.
- Una CuentaBancaria puede tener uno o muchos operadores.
- Una Persona puede tener cero a muchas CuentasBancarias.
- Una Persona puede operar cero a muchas CuentasBancarias.

```
@startuml
class Empresa
class Persona
class CuentaBancaria
Empresa "empleador 1" -- "empleado 7" Persona
Persona "propietario 1" -- "*" CuentaBancaria
Persona "operador 1..*" -- "*" CuentaBancaria
hide circle
@enduml
```

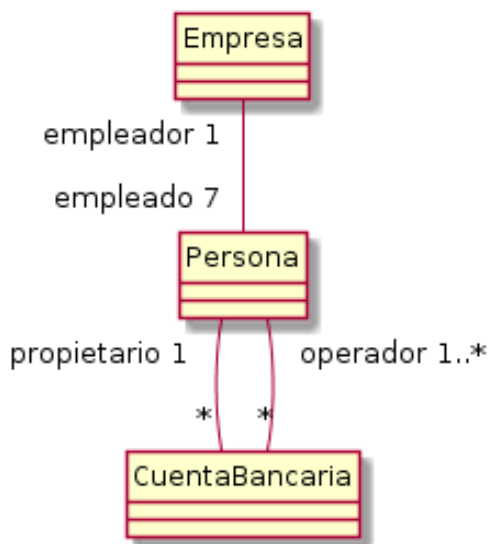


Figura 16.

Cuando se lee un modelo UML, es muy importante averiguar exactamente qué dice el modelo en lugar de realizar cualquier asunción o inventar la semántica. A esto lo llamamos leer el modelo literalmente.

Por ejemplo, la Figura 16 indica que una Empresa puede tener exactamente 7 empleados, ni más y ni menos. La mayoría de las personas considerarían que esta semántica es bastante

extraña o incluso incorrecta (a menos que sea una empresa muy extraña), pero esto es lo que dice el modelo.

Existe mucho debate acerca de si la multiplicidad se debería mostrar en modelos de análisis. Pensamos que se debería hacer porque la multiplicidad describe reglas de negocio, requisitos y restricciones y puede exponer suposiciones injustificadas realizadas sobre el negocio. Dichas asunciones se deben exponer tan pronto como sea posible.

Asociaciones reflexivas

Es bastante común para una clase tener una asociación consigo misma. Esto se denomina una asociación reflexiva y significa que los objetos de esa clase tienen vínculos a otros objetos de la misma clase (incluso consigo mismo). Un buen ejemplo de una asociación reflexiva se muestra en la Figura 17. Todo objeto Directorio puede tener vínculos a cero o más objetos Directorio que desempeñan el rol subdirectorio y a cero o un objeto Directorio que desempeña el rol padre. Además, todo objeto Directorio está asociado con cero o más objetos Archivo. Esto modela una estructura de directorio genérica bastante bien, aunque merece la pena mencionar que sistemas específicos de archivo (como Windows) pueden tener diferentes restricciones de multiplicidad para este modelo.

```
@startuml
class Directorio
class Archivo
Directorio "subdirectorio *" -- "padre 0..1" Directorio
note right on link
    Asociación reflexiva
end note
Directorio "1" -- "*" Archivo
hide circle
@enduml
```

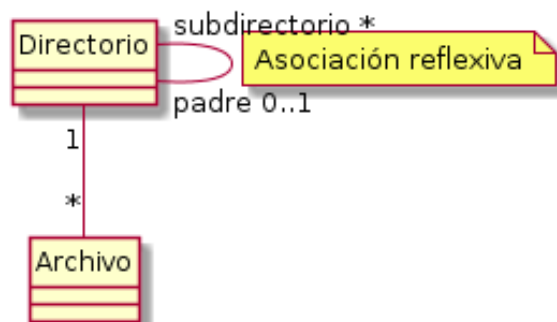


Figura 17.

La Figura 18 muestra un diagrama de objeto de ejemplo que concuerda con el diagrama de clase de la Figura 17.

```

@startuml
object "<u>c:Directorio</u>" as c
object "<u>windows:Directorio</u>" as windows
object "<u>misDocumentos:Directorio</u>" as misDocumentos
object "<u>system:Directorio</u>" as system
object "<u>config.ini:Archivo</u>" as config
object "<u>modelo.uml:Archivo</u>" as modelo
object "<u>unidad5.pdf:Archivo</u>" as unidad5
c -- windows
c -- misDocumentos
windows -- system
c -- config
misDocumentos -- modelo
misDocumentos -- unidad5
@enduml

```

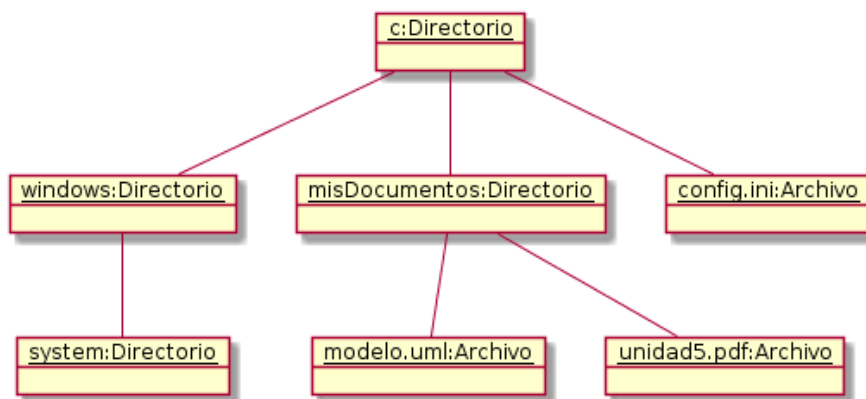


Figura 18.

Jerarquías y redes

Cuando se modela, encontrará que los objetos se organizan en jerarquías o redes. Una jerarquía tiene un objeto raíz y cualquier otro nodo en la jerarquía tiene exactamente un objeto directamente sobre él. Los árboles de directorio forman jerarquías. Lo mismo lo hacen elementos en XML y documentos HTML. La jerarquía es una forma ordenada y estructura y en cierto sentido rígida de organizar objetos. Un ejemplo se muestra en la Figura 19. En la Figura 20 se ilustra un diagrama de objetos, instancia del modelo de la Figura 19.

```

@startuml
class A
A "*" -- "0..1" A
note right on link
    Asociación jerarquía
end note

```

hide circle

@enduml

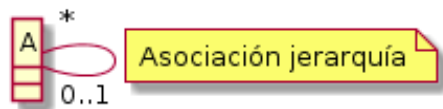


Figura 19.

@startuml

object "<u>a1:A</u>" as a1

object "<u>b1:A</u>" as b1

object "<u>c1:A</u>" as c1

object "<u>d1:A</u>" as d1

object "<u>e1:A</u>" as e1

object "<u>f1:A</u>" as f1

object "<u>g1:A</u>" as g1

a1 -- b1

a1 -- c1

a1 -- d1

b1 -- e1

d1 -- f1

d1 -- g1

@enduml

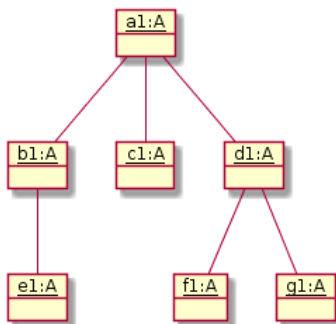


Figura 20.

Sin embargo, en una red no hay objetos raíz, aunque esto no está excluido. El modelo de red se ilustra en la Figura 21. En las redes, cada objeto puede tener muchos objetos directamente conectados a él. No existe un concepto real de arriba o abajo en una red. Es una estructura mucho más flexible en la que es posible que ningún nodo tenga primacía sobre otro. La World Wide Web forma una red compleja de nodos, como se ilustra de forma sencilla en la Figura 22.

@startuml

class A

A "*" -- "*" A

note right on link

Asociación - red

end note

hide circle

@enduml



Figura 21.

@startuml

object "<u>a1:A</u>" as a1

object "<u>b1:A</u>" as b1

object "<u>c1:A</u>" as c1

object "<u>d1:A</u>" as d1

object "<u>e1:A</u>" as e1

object "<u>f1:A</u>" as f1

object "<u>g1:A</u>" as g1

g1 -- b1

d1 -- a1

a1 -- c1

f1 -- a1

c1 -- g1

d1 -- e1

d1 -- b1

@enduml

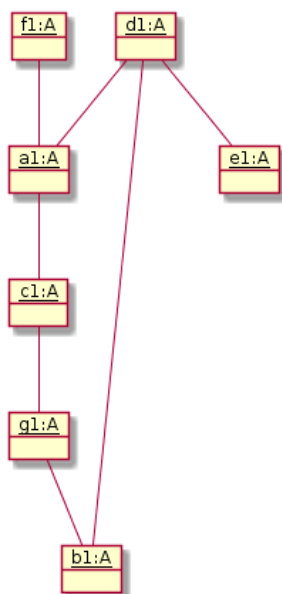


Figura 22.

Como ejemplo para ilustrar jerarquías y redes, consideremos productos. Existen dos abstracciones fundamentales:

- TipoProducto, un tipo de producto como una impresora de tinta.
- ElementoProducto, una impresora específica, con número de serie 0001123430.

Los TipoProducto a menudo tienden a formar redes, por lo que un TipoProducto como un ordenador puede constar de una CPU, pantalla, teclado, ratón, tarjeta gráfica y otros TiposProducto. Cada uno de estos TiposProducto describe un tipo de producto, no un elemento individual, y estos tipos de producto pueden participar en otros TiposProducto como diferentes componentes de ordenador.

Sin embargo, si consideramos los ElementosProducto, que son instancias específicas de un TipoProducto, cualquier ElementoProducto, como una CPU específica, solamente se puede vender y distribuir una vez como parte de un paquete de artículos. Los Elementas Producto forman jerarquías.

Navegabilidad

La navegabilidad nos muestra que es posible pasar desde un objeto de la clase fuente a uno o más objetos de la clase destino, dependiendo de la multiplicidad. En la Figura 23 los objetos Pedido conocen a los objetos Producto pero no viceversa.

```
@startuml
class Pedido
class Producto
Pedido "*" --> "*" Producto
hide circle
@enduml
```

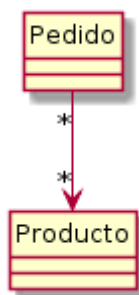


Figura 23.

Uno de los objetivos al modelar es minimizar el acoplamiento entre clases, y utilizando la navegabilidad, es una buena forma de hacerlo. Sin embargo, es una práctica habitual en el diseño y no en el análisis.

Al hacer unidireccional la asociación entre Pedido y Producto, puede navegar fácilmente desde objetos Pedido a objetos Producto, pero no hay navegabilidad inversa desde objetos Producto

a objetos Pedido. Por lo tanto, los objetos Producto no saben que pueden estar participando en un Pedido en particular y, por lo tanto, no tienen acoplamiento con Pedido.

La navegabilidad se muestra al anexar una punta de flecha en un extremo de la relación como se muestra en la Figura 23.

En la Figura 24 se ilustra una asociación bidireccional, donde ambos sentidos son navegables: A a B es navegable, y B a A es navegable.

```
@startuml
class A
class B
A -- B
note right on link
  Asociación bidireccional
  A a B es navegable
  B a A es navegable
end note
hide circle
@enduml
```

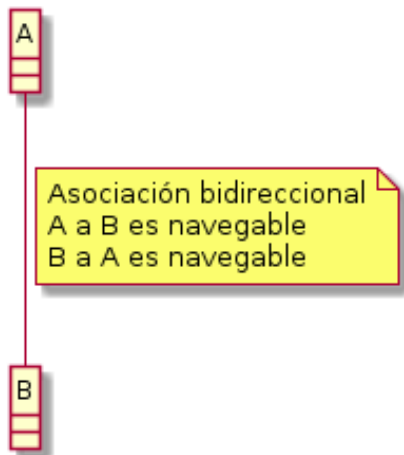


Figura 24.

En la Figura 25 se ilustra una asociación unidireccional, navegable en un solo sentido: A a B es navegable, pero desde B a A no es navegable.

```
@startuml
class A
class B
A --> B
note right on link
  Asociación unidireccional
end note
hide circle
@enduml
```

```

A a B es navegable
B a A no es navegable
end note
hide circle
@enduml

```

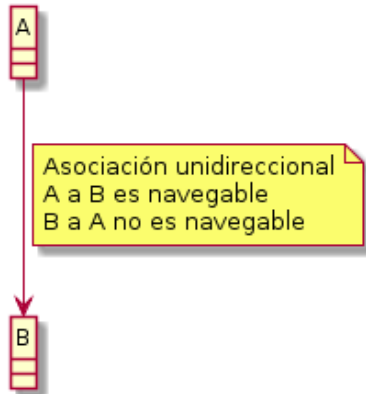


Figura 25.

De todos modos, si una asociación no es navegable en una determinada dirección, puede ser posible atravesar la relación en esa dirección. Sin embargo, el coste computacional será muy alto. En el ejemplo en la Figura 23, aunque no puede navegar de forma inversa desde Producto a Pedido, puede encontrar el objeto Pedido asociado con un determinado objeto Producto al buscar por todos los objetos Pedido en turno. Ha atravesado entonces una relación no navegable, pero a un coste computacional alto. La navegabilidad en un sentido es como una calle de un solo sentido; podría llegar al final de ésta por medio alguna otra ruta (más larga).

Si existe un nombre de rol en el extremo destino de la relación, los objetos de la clase origen pueden hacer referencia a objetos de la clase destino al utilizar este nombre de rol.

En términos de implementación en lenguajes orientados a objetos, la navegabilidad implica que el objeto origen contiene una referencia de objeto al objeto destino. El objeto fuente puede utilizar esta referencia de objeto para enviar mensajes al objeto destino.

Asociaciones y atributos

Existe un vínculo muy próximo entre asociaciones y atributos. Una asociación entre una clase origen y una clase destino significa que los objetos de una clase origen pueden albergar una referencia de objeto a objetos de la clase destino. Otra forma de ver esto es que una asociación es equivalente a la clase origen que tiene un pseudo-atributo de la clase destino. Un objeto de la clase origen puede hacer referencia a un objeto de la clase destino al utilizar este pseudo atributo; véase Figura 26.

No existe ningún lenguaje de programación orientado a objetos comúnmente utilizado que tenga un constructor específico de lenguaje para soportar asociaciones. Por lo tanto, cuando el código se genera automáticamente desde un modelo UML, las asociaciones uno a uno se convierten en atributos de la clase origen, obteniéndose el modelo de la Figura 27.

```

@startuml
class Casa
note right: origen
class Direccion
note right: destino
Casa "1"-->"direccion 1" Direccion
note right on link
    Si una asociación navegable tiene un nombre de rol,
    entonces es como si la clase origen tiene un atributo
    con el mismo nombre que el nombre de rol
    y el mismo tipo que la clase destino.
end note
hide members
hide circle
@enduml

```

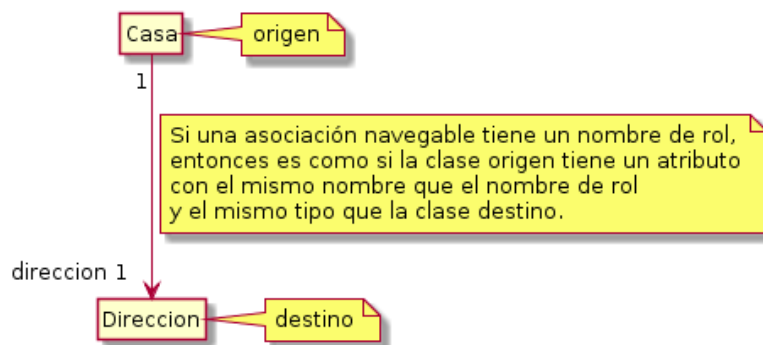


Figura 26.

```

@startuml
class Casa {
    direccion: Direccion
}
hide circle
@enduml

```

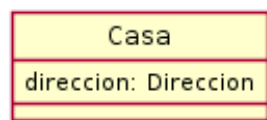


Figura 27.

En la Figura 27 la clase Casa contiene un atributo denominado dirección que es del tipo Dirección. Observe cómo el nombre de rol proporciona el nombre de atributo y la clase en el extremo de la asociación proporciona el tipo de atributo. El código Java a continuación se ha generado a partir del modelo en la Figura 27.

```
public class Casa {
    private Direccion dirección;
}
```

Las multiplicidades destino mayores que 1 se implementan como:

- Un atributo del tipo array (un constructor que se soporta en la mayoría de los lenguajes), o bien
- Un atributo de algún tipo que es una colección.

Las colecciones son clases cuyas instancias tienen el comportamiento especializado de poder almacenar y recuperar referencias a otros objetos. Un ejemplo Java común de una colección es un Vector, pero existen muchas más.

Utilice asociaciones solamente cuando la clase destino es una parte importante del modelo. De lo contrario, modele la relación al utilizar atributos. Las clases importantes son clases de negocio que describen parte del dominio de negocio. Las clases generales, utilizadas en cualquier dominio, son componentes de biblioteca (por ejemplo: las clases Integer, String, Date, Time), y generalmente son incorporados como tipo de datos en los atributos.

Hasta cierto sentido, la opción de asociaciones explícitas versus atributos es una cuestión de estilo. El mejor enfoque siempre es uno en el que el modelo y los diagramas expresan el problema claramente y de forma precisa. A menudo, es más claro mostrar una asociación a otra clase que modelar la misma relación como un atributo que sería más difícil de ver. Cuando la multiplicidad destino es mayor que 1, esto es una buena indicación de que el destino es importante para el modelo, por lo tanto, utiliza asociaciones para modelar la relación.

Si la multiplicidad destino es exactamente 1, el objeto destino puede ser una parte de la fuente y por lo tanto no merece la pena mostrarse como una asociación, se puede modelar mejor como un atributo. Esto es especialmente cierto si la multiplicidad es exactamente 1 en ambos extremos de la relación (como en la Figura 26) donde ni la fuente ni el destino pueden existir solos.

Clases de asociación

Un problema común en el modelado orientado a objetos es el siguiente: cuando tiene una relación muchos a muchos entre dos clases, a veces existen algunos atributos que no se pueden acomodar fácilmente en ninguna de las clases. Podemos ilustrar esto al considerar el ejemplo en la Figura 28.

```
@startuml
class Empresa
class Persona
Empresa "*" -- "*" Persona
hide circle
@enduml
```



Figura 28.

A primera vista esto parece un modelo bastante inocuo:

- Todo objeto Persona puede trabajar para muchos objetos Empresa.
- Todo objeto Empresa puede emplear muchos objetos Persona.

Sin embargo, ¿qué sucede si añade la regla de negocio que cada Persona tiene un sueldo por cada Empresa en la que esté empleado? ¿Dónde se debería almacenar el sueldo, en la clase Persona o en la clase Empresa?

No puede hacer que el sueldo de Persona sea un atributo de la clase Persona, ya que cada instancia Persona puede trabajar para muchas Empresas y puede tener un sueldo diferente con cada Empresa. De forma similar, no puede hacer que el sueldo de Persona sea un atributo de Empresa, ya que cada instancia Empresa emplea muchas Personas, todas con sus sueldos diferentes. La respuesta es que el sueldo es una propiedad de la propia asociación. Para cada asociación de empleo que un objeto Persona tiene con un objeto Empresa, existe un sueldo específico.

UML le permite modelar esta situación con una clase de asociación como se muestra en la Figura 29. Es importante entender esta sintaxis; muchas personas creen que la clase de asociación es un cuadro que cuelga de la asociación. Sin embargo, nada más lejos de la realidad. La clase de asociación es en realidad la línea de asociación (incluidos todos los nombres de rol y multiplicidades), la línea de puntos que desciende y el cuadro de clase en el extremo de la línea de puntos. Es decir, todo el conjunto, todo lo que se muestra en el área indicada.

```
@startuml
```

```
class Empresa
```

```
class Persona
```

```
Empresa "*" -- "*" Persona
```

```
(Empresa, Persona) .. Trabajo
```

```
class Trabajo {
```

```
    sueldo: double
```

```
}
```

```
note right on link
```

```
    clase de asociación
```

```
    la clase de asociación consta de:
```

```
    la clase, la asociación y la línea discontinua
```



```

class Empresa
class Persona
class Trabajo {
    sueldo: double
}
Empresa "1"--"*" Trabajo
Trabajo "*"--"1" Persona
hide circle
@enduml

```

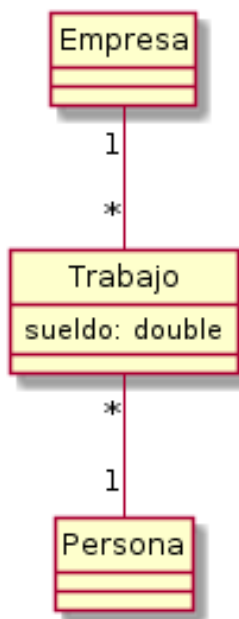


Figura 30.

Dependencia

Una dependencia indica una relación entre dos o más elementos de modelo donde un cambio en un elemento (el proveedor) puede afectar o proporcionar la información necesaria por el otro elemento (el cliente).

En otras palabras, el cliente depende en cierto sentido del proveedor. Utilizamos dependencias para modelar relaciones entre clasificadores donde un clasificador depende del otro de alguna forma, pero la relación no es realmente una asociación o generalización.

Por ejemplo, puede pasar un objeto de una clase como un parámetro a una operación de un objeto de una clase diferente. Existe cierto tipo de relación entre las clases de esos objetos, pero no es realmente una asociación. Puede utilizar la relación de dependencia como algo general para modelar este tipo de relación, en la Figura 31 se ilustra esta situación, el método metodo1 posee el parámetro b: B_Proveedor. Otras situaciones se presentan en casos donde una operación (metodo2 en Figura 31) devuelve un objeto instancia de la clase proveedora (B_Proveedor). Incluso, es posible que una operación del cliente (hacerAlgo()) en Figura 31)

utiliza un objeto instancia de la clase proveedora en algún lugar de su implementación, por ejemplo, el siguiente fragmento en Java:

```
class A_Cliente {
...
void hacerAlgo() {
    B_Proveedor unObjeto= new B_Proveedor();
    // Utiliza unObjeto de alguna forma
    ....
}
}
```

```
@startuml
class A_Cliente {
    metodo1(b: B_Proveedor)
    metodo2(): B_Proveedor
    hacerAlgo()
}
note right
    cliente
end note
class B_Proveedor{
}
note right
    proveedor
end note
A_Cliente --> B_Proveedor
hide circle
@enduml
```

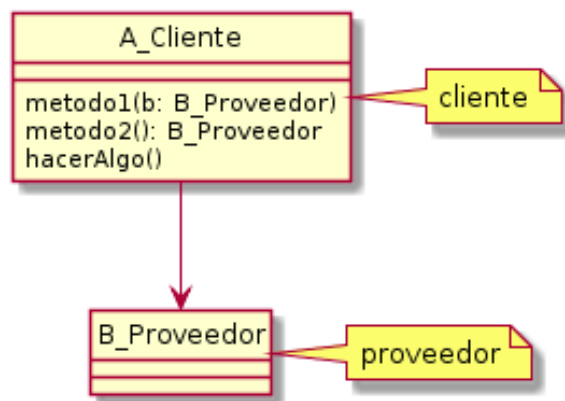


Figura 31.

Generalización

La generalización es una relación entre un elemento más general y un elemento más específico, donde el elemento más específico es totalmente coherente con el elemento más general, pero contiene más información.

Los dos elementos obedecen al principio de sustitución; podemos utilizar el elemento más específico en cualquier lugar que se espere el elemento más general sin romper el sistema.

Claramente, la generalización implica el nivel más alto de dependencia (y, por lo tanto, acoplamiento) entre dos elementos.

Conceptualmente, la generalización es una idea sencilla. Ya está familiarizado con la noción de elementos generales como un árbol, y luego elementos más específicos como un roble que es un tipo particular de árbol.

La generalización se aplica a todos los clasificadores. Ya ha visto la generalización aplicada a casos de uso y actores, y ahora verá cómo se aplica a clases. En la Figura 32 tenemos una clase denominada Forma; ésta es una noción muy general. De ella, derivamos hijos, subclases, descendientes (todos estos términos son de uso común) que son variantes más específicas de la idea general de Forma. Por el principio de sustitución, podemos utilizar una instancia de cualquiera de estas subclases en cualquier lugar que se espere una instancia de la súper clase Forma.

@startuml

class Forma

note right

Elemento más general

padre

superclase

end note

class Cuadrado

class Circulo

class Rectangulo

note bottom

Elemento más específico

hijo

subclase

end note

Forma <|-- Cuadrado

Forma <|-- Circulo

Forma <|-- Rectangulo

note right on link

es un tipo de

end note

note as n1

Una jerarquía de generalización

end note

hide members

hide circle

@enduml

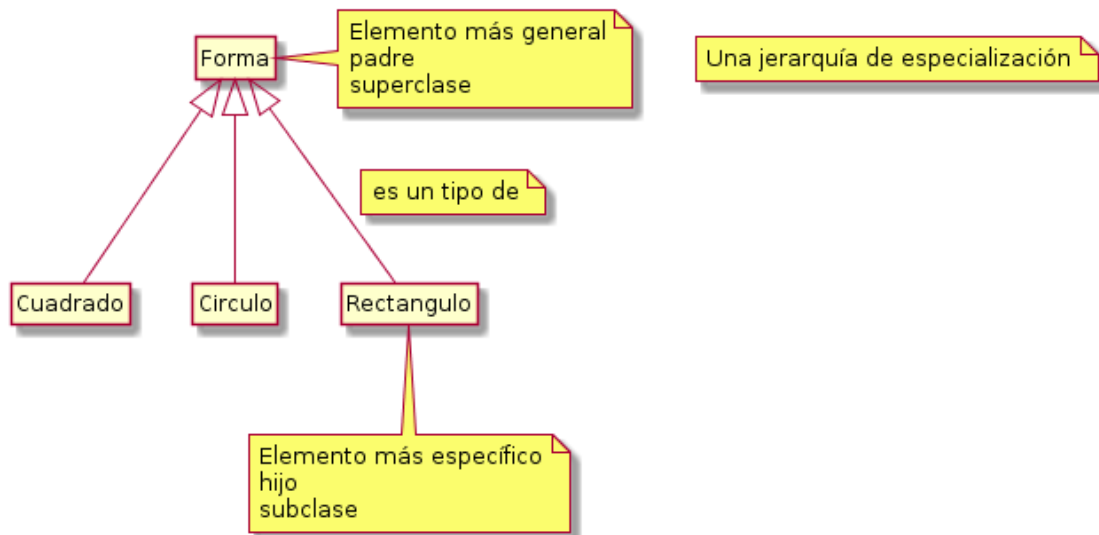


Figura 32.

Como verá cuando examinemos los atributos y operaciones detallados de estas clases, podemos llegar a la jerarquía anterior de dos formas: por medio de un proceso de especialización o un proceso de generalización. En especialización, identificaríamos primero el concepto general de Forma en análisis y luego lo especializaríamos en tipos específicos de forma. En generalización, identificaríamos los Cuadrado, Círculo y Triángulo más especializados en análisis y luego observaríamos que tienen características comunes que podríamos resolver en una súper clase más general. Los analistas orientados a objetos tienden a utilizar la especialización y generalización al mismo tiempo.

Herencia de clase

Cuando organiza clases en una jerarquía de generalización, como se muestra en la Figura 32, implícitamente tiene herencia entre las clases donde las subclases heredan todas las características de sus súper clases. Para ser más específico, las súper clases heredan:

- Atributos
- Operaciones
- Relaciones
- Restricciones

Las subclases también pueden añadir nuevas características y redefinir operaciones de superclase.

En el ejemplo en la Figura 33, las subclases de Forma, Cuadrado y Círculo heredan todos los atributos, operaciones y restricciones de la superclase Forma. Esto significa que aunque no ve estas características en las subclases, están allí implícitamente. Decimos que Cuadrado y Círculo son tipos de Forma.

```
@startuml
class Forma {
    origen: Punto
    ancho: int
    alto: int
    dibujar(g: Grafico)
    obtenerArea(): int
    obtenerAreaLimite(): int
}
class Cuadrado
class Circulo {
    radio: int
}
Forma <|-- Cuadrado
Forma <|-- Circulo
note "{ancho = alto}" as n1
Cuadrado .. n1
Circulo .. n1
hide circle
@enduml
```

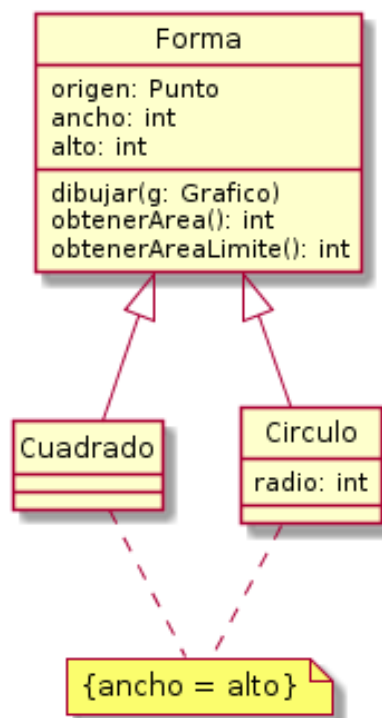


Figura 33.

Observe que las operaciones dibujar () y obtenerArea () en Forma pueden no ser apropiadas para las subclases. Esperaría un objeto Cuadrado, cuando se envía el mensaje dibujar () , para dibujar un cuadrado, y un objeto Círculo, cuando envía el mensaje dibujar () , para dibujar un círculo. La operación dibujar() predeterminada que ambas subclases han heredado de su padre claramente no lo harán. De hecho, esta operación puede no dibujar nada en absoluto, después de todo, ¿cuál debería ser el aspecto de una Forma? Los mismos argumentos se aplican a obtenerArea() . ¿Cómo calcula el área de una Forma?

Estos problemas apuntan a la necesidad de que las subclases puedan cambiar el comportamiento de la superclase. Cuadrado y Circulo necesitan implementar sus propias operaciones dibujar() y obtenerArea() que redefinen las operaciones predeterminadas facilitadas por el padre y proporcionan un comportamiento más apropiado. La Figura 34 muestra esto: las subclases Cuadrado y Circulo han proporcionado sus propias operaciones dibujar() y obtenerArea() que tienen los comportamientos apropiados.

- Cuadrado::dibujar(g: Grafico). Dibuja un cuadrado.
- Cuadrado::obtenerArea(): int. Calcula y devuelve el área del cuadrado.
- Circulo::dibujar (g : Grafico). Dibuja un círculo.
- Circulo::obtenerArea (): int. Calcula y devuelve el área del círculo.

@startuml

```

abstract class Forma {
    origen: Punto
    ancho: int
    alto: int
    {abstract} dibujar(g: Grafico)
    {abstract} obtenerArea(): int
    obtenerAreaLimite(): int
}
note right {
    Clase abstracta
    dibujar() y obtenerArea() son operaciones abstractas
    obtenerAreaLimite() es una operación concreta
}
class Cuadrado {
    dibujar(g: Grafico)
    obtenerArea(): int
}
class Circulo {
    radio: int
    dibujar(g: Grafico)

```

```

    obtenerArea(): int
}
Forma <|-- Cuadrado
Forma <|-- Circulo
note "Clases concretas \ndibujar() y obtenerArea() son operaciones concretas" as n1
Cuadrado .. n1
Circulo .. n1
hide circle
@enduml

```

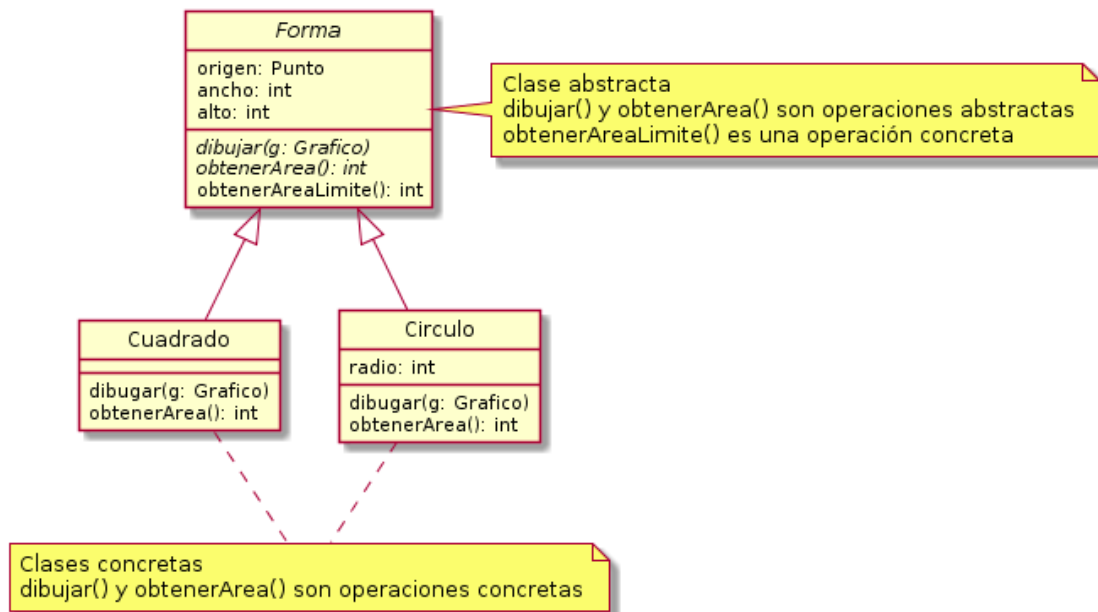


Figura 34.

Para redefinir una operación de superclase, una subclase debe proporcionar una operación con exactamente la misma firma/signatura que la operación de superclase que desea redefinir. UML define la firma (signatura) de operación como el nombre de operación, su tipo de retorno y los tipos de todos los parámetros listados en orden. Los nombres de parámetro no cuentan, ya que son una forma conveniente de hacer referencia a un parámetro específico dentro de un cuerpo de operación y por lo tanto no son parte de la firma.

Algunas veces preferiríamos diferir la implementación de una operación a las subclases. En nuestro ejemplo Forma, la operación Forma::dibujar(g: Grafico) es un caso a destacar. No podemos proporcionar ninguna implementación sensata de esta operación en la propia clase Forma, ya que no sabemos cómo se deberían to dibujar las formas; el concepto de dibujar una forma es demasiado abstracto para tener cualquier implementación concreta.

Puede especificar que una operación carezca de una implementación al convertirla en una operación abstracta. En UML hace esto al escribir el nombre de la operación en cursiva (Figura 34, precedido la declaración de la operación por {abstract} en plantUML). Cuando piensa en ello, una clase con una o más operaciones abstractas está incompleta ya que hay algunas operaciones que no tienen una implementación. Esto significa que no puede instanciar dichas

clases y, por lo tanto, se conocen como clases abstractas. Escriba el nombre de la clase en cursiva para mostrar que es abstracta (Figura 34, en plantUML se define la clase como `abstract class`).

En el ejemplo en la Figura 34 tenemos la clase abstracta *Forma* que tiene dos operaciones abstractas: *Forma::dibujar(g: Grafico)* y *Forma::obtenerArea() :int*. Las implementaciones de estas operaciones están proporcionadas por las subclases *Cuadrado* y *Circulo*. Aunque *Forma* está incompleta y no se puede instanciar, sus subclases proporcionan las implementaciones que faltan, están completas y se pueden instanciar. Cualquier clase que se puede instanciar se conoce como clase concreta.

La operación *obtenerAreaLimite()* (Figura 34) es una operación concreta de *Forma* porque el área límite de cada tipo de *Forma* se calcula de igual forma, siempre es el ancho de la forma multiplicado por su alto.

Existen un par de ventajas de utilizar clases y operaciones abstractas.

- Puede definir un conjunto de operaciones abstractas en la superclase abstracta que se deben implementar por todas las subclases *Forma*. Puede pensar en esto como definir un contrato que todas las subclases concretas *Forma* deben implementar.
- Puede escribir código para manipular *Formas* y luego sustituir *Circulo*, *Cuadrado* y otras subclases *Forma* según sea apropiado. Según el principio de sustitución, el código escrito para manipular *Formas* debería funcionar para todas las subclases *Forma*.

Examinaremos estas ventajas en más detalle cuando tratemos el polimorfismo más adelante.

Antes de llegar al polimorfismo, es una buena idea entender algo sobre los niveles de abstracción. ¿Qué está mal en el modelo de la Figura 35?

La respuesta es los niveles de abstracción. Una jerarquía de generalización define un conjunto de niveles de abstracción desde el más general en la parte superior, al más específico en la parte inferior. Siempre debería tratar de mantener un nivel uniforme de abstracción en cada nivel de la jerarquía de generalización. En el ejemplo anterior no hemos conseguido esto. *JaguarXJS* es un tipo de coche. Claramente éste es un nivel de abstracción más bajo que *Camión*. Puede ajustar el modelo de forma fácil al introducir una superclase *Automovil* entre *JaguarXJS* y *Vehículo*.

```
@startuml
class Vehiculo
class JaguarXLS
class Camion
Vehiculo <|-- JaguarXLS
Vehiculo <|-- Camion
hide circle
hide members
@enduml
```

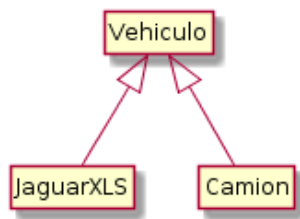


Figura 35.

UML permite que una clase tenga más de una superclase directa. Esto se denomina herencia múltiple. La superclase hereda de todas sus superclases directas. Esto lo trataremos mas adelante.

Polimorfismo

Polimorfismo significa muchas formas. Una operación polimórfica es una que tiene muchas implementaciones. Ya ha visto dos operaciones polimórficas en el ejemplo Forma. Las operaciones abstractas dibujar() y obtenerArea() en la clase Forma tienen dos implementaciones diferentes, una implementación en la clase Cuadrado y una implementación diferente en la clase Circulo. Las operaciones tienen muchas formas y son por lo tanto polimórficas.

La Figura 36 ilustra el polimorfismo perfectamente. Definimos una clase abstracta Forma con operaciones abstractas dibujar() y obtenerArea().

@startuml

```

abstract class Forma {
    origen: Punto
    ancho: int
    alto: int
    {abstract} dibujar(g: Grafico)
    {abstract} obtenerArea(): int
    obtenerAreaLimite(): int
}
class Cuadrado {
    dibujar(g: Grafico)
    obtenerArea(): int
}
class Circulo {
    radio: int
    dibujar(g: Grafico)
    obtenerArea(): int
}
Forma <|-- Cuadrado
Forma <|-- Circulo
  
```



```

class Lienzo
Lienzo "1"--"*" Forma
hide circle
@enduml

```

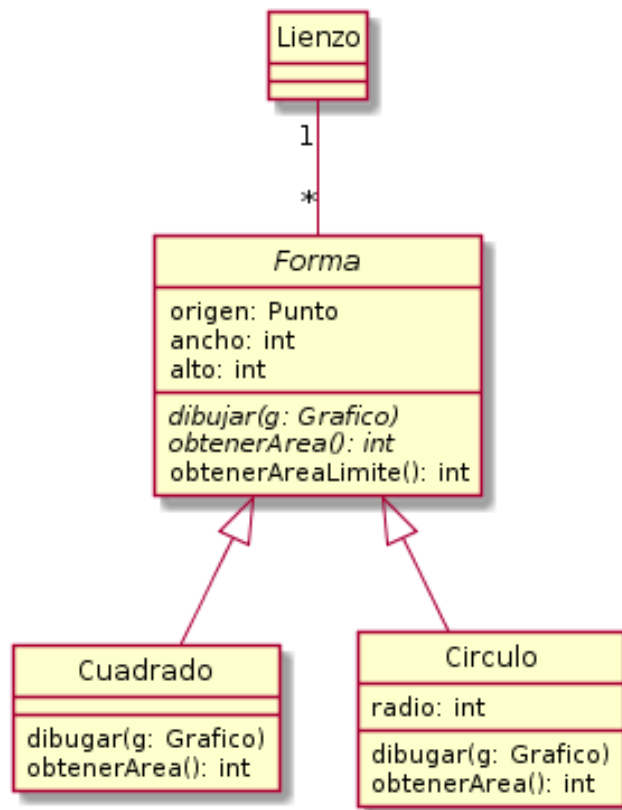


Figura 36.

Cuadrado y Circulo heredan de Forma y proporcionan implementaciones para las operaciones polimórficas `Forma::dibujar()` y `Forma::obtenerArea()`. Todas las subclases concretas de Forma deben proporcionar operaciones `dibujar()` y `obtenerArea()` concretas, porque son abstractas en la superclase. Esto significa que para `dibujar()` y `obtenerArea()` puede tratar todas sus subclases de Forma de forma similar. Un conjunto de operaciones abstractas es una forma de definir un conjunto de operaciones que todas las subclases concretas deben implementar. Esto se conoce como un contrato.

Claramente la implementación de `dibujar()` y `obtenerArea()` será diferente para Cuadrados y Círculos. La operación `dibujar()` dibujará un cuadrado para objetos de la clase Cuadrado y dibujará un círculo para objetos de la clase Circulo. Puede ver que la operación `obtenerArea()` también tendrá diferentes implementaciones. Devolverá `ancho*alto` para un cuadrado y `Pi por radio al cuadrado` para un círculo. Esta es la esencia del polimorfismo; los objetos de clases diferentes tienen operaciones con la misma firma pero diferentes implementaciones.

Encapsulación, herencia y polimorfismos son los tres pilares de la orientación a objetos. El polimorfismo le permite diseñar sistemas más sencillos que pueden acomodar más fácilmente el cambio porque le permite tratar objetos diferentes de la misma forma.

De hecho, lo que hace que el polimorfismo sea un aspecto esencial de la orientación a objetos, es que le permite enviar a objetos de clases diferentes el mismo mensaje y hacer que los objetos respondan en consecuencia. Por lo tanto, si envía a objetos de la clase Cuadrado el mensaje dibujar() , dibujarán un cuadrado, y si envía a objetos de la clase Circulo el mismo mensaje, dibujarán un círculo. Los objetos parecen presentar un tipo de inteligencia.

Aquí tiene un ejemplo de polimorfismo en acción. Suponga que tiene una clase Lienzo que mantiene una colección de Formas. Aunque ésta es una imagen algo simplificada, muchos sistemas gráficos funcionan de esta forma. El modelo para este sistema gráfico se muestra en la Figura 36.

Ahora, sabe que no puede crear una instancia de Forma (porque es abstracta) pero según el principio de sustitución, puede crear instancias de sus subclases concretas y sustituir éstas en cualquier lugar donde se invoca a una Forma.

Por lo tanto, aunque la Figura 36 muestra que los objetos del tipo Lienzo contienen una colección de muchos objetos Forma, los únicos objetos que puede situar en la colección son instancias de subclases concretas de Forma porque Forma es abstracta y no se puede instanciar. En este caso, existen dos subclases concretas, Circulo y Cuadrado, por lo tanto, la colección puede contener objetos Circulo y/o objetos Cuadrado.

En la Figura 37 hemos creado un modelo de objeto del diagrama de clase en la Figura 36. Este modelo de objeto muestra que un objeto :Lienzo contiene una colección de cuatro objetos Forma, s1, s2, s3 y s4, donde s1, s3 y s4 son objetos de clase Circulo y s2 es un objeto de clase Cuadrado.

```
@startuml
object "<u>:Lienzo</u>" as unLienzo
object "<u>s1:Circulo</u>" as s1
object "<u>s2:Cuadrado</u>" as s2
object "<u>s3:Circulo</u>" as s3
object "<u>s4:Circulo</u>" as s4
unLienzo -- s1
unLienzo -- s2
unLienzo -- s3
unLienzo -- s4
@enduml
```

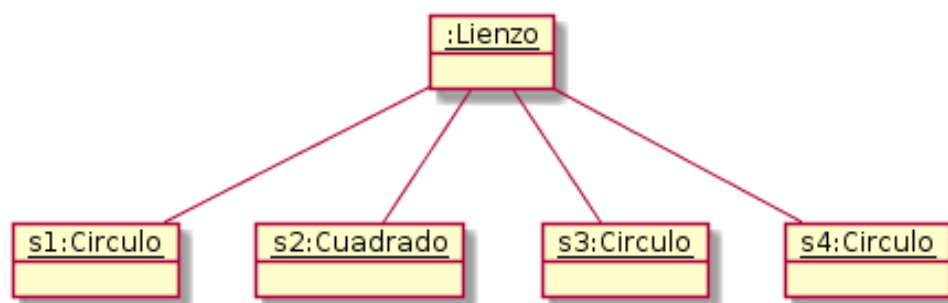


Figura 37.

¿Qué sucede cuando el objeto :Lienzo itera por la colección de Formas y envía a cada objeto en la colección el mensaje dibujar()? Cada objeto realiza lo correcto, los objetos Cuadrado dibujan cuadrados y los objetos Círculo dibujan círculos. Es la clase del objeto la que determina qué dibuja el objeto; es decir, la clase del objeto determina la semántica de del conjunto de operaciones que ofrece el objeto. En la Figura 38 se ilustra la secuencia de mensajes empleando un diagrama de secuencia de mensajes.

@startuml

participant "<u>s1:Circulo</u>" as s1

participant "<u>s2:Cuadrado</u>" as s2

participant "<u>s3:Circulo</u>" as s3

participant "<u>s4:Circulo</u>" as s4

unLienzo -> s1 :dibujar(this)

unLienzo -> s2 :dibujar(this)

unLienzo -> s3 :dibujar(this)

unLienzo -> s4 :dibujar(this)

@enduml

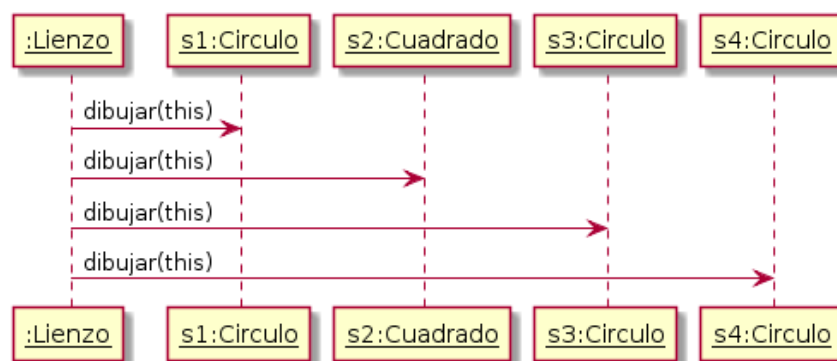


Figura 38.

El punto clave aquí es que cada objeto responde a un mensaje al invocar la operación correspondiente especificada por su clase. Todos los objetos de la misma clase responderán al mismo mensaje al invocar la misma operación. Esto no significa necesariamente que todos los objetos de la misma clase respondan al mismo mensaje exactamente en la misma forma. Los resultados de una invocación de operación a menudo dependen del estado del objeto; los valores de todos sus atributos y el estado de todas sus relaciones. Por ejemplo, podría tener tres objetos de clase Cuadrado con diferentes valores de atributo para ancho y alto. Cuando envía el mensaje dibujar() a cada uno de estos objetos en turno, cada uno de ellos dibujará un cuadrado (el significado o semántica de la operación permanece igual), pero cada cuadrado tendrá un tamaño diferente dependiendo de los valores de atributo de ancho y alto.

Aquí tiene otro ejemplo. Las reglas de negocio sobre retirar dinero y calcular el interés son diferentes dependiendo del tipo de cuenta bancaria. Por ejemplo, las cuentas corrientes

tienden a tener un límite al descubierto y por lo tanto pueden tener un saldo negativo, mientras que las cuentas caja de ahorro no permitirán que el saldo caiga por debajo de 0. De forma similar, el interés a menudo se calcula y acumula en la cuenta de forma diferente. Una forma sencilla de modelar esto se muestra en la Figura 39. Se define una clase abstracta Cuenta y luego se proporcionan subclases concretas CuentaCorriente, CajaDeAhorro. La clase abstracta define operaciones abstractas para retirar() y calcularIntereses() y están implementadas de formas diferentes por cada una de las subclases concretas.

```
@startuml
abstract class Cuenta {
    {abstract} retirar(cantidad)
    {abstract} calcularIntereses()
    depositar(cantidad)
}
class CajaDeAhorro {
    retirar(cantidad)
    calcularIntereses()
    depositar(cantidad)
}
class CuentaCorriente {
    retirar(cantidad)
    calcularIntereses()
}
Cuenta <|-- CajaDeAhorro
Cuenta <|-- CuentaCorriente
class Banco
Banco "1" -- "*" Cuenta
hide circle
@enduml
```


Un conjunto de generalización agrupa subclases de acuerdo a una regla particular, o base de especialización. Por ejemplo, Forma podría ser clasificado en dos grupos distintos de Formas: formas bidimensionales y formas tridimensionales.

Puede indicar esta partición de subclases en un diagrama de clases al asociar cada grupo de formas con un conjunto de generalización diferente. El nombre de conjunto de generalización se ubica en el conjunto de relaciones de generalización/especialización, esta característica no está soportada en PlantUML, por eso es agregado en cada relación generalización en la Figura 40.

```
@startuml
abstract class Forma
class Cuadrado
class Circulo
class Triangulo
class Cubo
class Esfera
class Piramide
Forma <|-- Cuadrado : 2D
Forma <|-- Circulo : 2D
Forma <|-- Triangulo : 2D
Forma <|-- Cubo : 3D
Forma <|-- Esfera : 3D
Forma <|-- Piramide : 3D
hide circle
hide members
@enduml
```

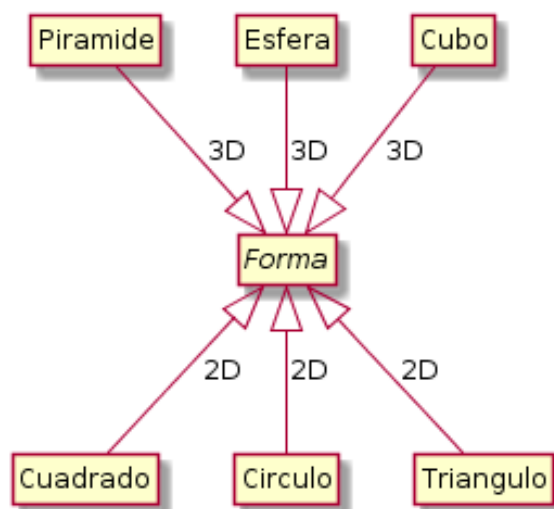


Figura 40.

Los conjuntos de generalización pueden tener restricciones aplicadas. Estas restricciones determinan si los conjuntos son:

- **{complete}**: Las subclases en el conjunto de generalización abarcan todas las posibilidades. Por ejemplo, un conjunto de generalización género de clase Persona que contiene las dos subclases Masculino y Femenino se podría considerar que están **{complete}** dado que solamente reconoce dos géneros.
- **{incomplete}**: Puede haber subclases que no sean éstas en el conjunto de generalización. El conjunto de generalización 2D está claramente **{incomplete}** puesto que hay potencialmente muchas Formas 2D.
- **{disjoint}**: Un objeto puede ser una instancia de uno y solamente uno de los miembros del conjunto de generalización. Este es el caso más común.
- **{overlapping}**: Un objeto puede ser una instancia de más de uno de los miembros del conjunto de generalización. Esto requiere herencia múltiple y/o clasificación múltiple.

Las restricciones del conjunto de generalización se pueden combinar como se muestra en la Tabla 9. La Figura 41 ilustra restricciones del conjunto de generalización aplicadas al ejemplo Forma, cabe destacar que PlantUML no soporta conjunto de generalización, por lo que no es posible aplicar estas restricciones, por tal motivo lo incorporamos como detalle de la relación de generalización.

Tabla 9.

Restricción	El conjunto está completo	Miembros del conjunto pueden tener instancias en común
{incomplete, disjoint}	No	No
{complete, disjoint}	Si	No
{incomplete, overlapping}	No	Si
{complete, overlapping}	Si	Si

```
@startuml
abstract class Forma
class Cuadrado
class Circulo
class Triangulo
Forma <|-- Cuadrado : 2D
Forma <|-- Circulo : 2D
Forma <|-- Triangulo : 2D {disjoint, incomplete}
class Cubo
class Esfera
class Piramide
Forma <|--up- Cubo : 3D {disjoint, incomplete}
Forma <|--up- Esfera : 3D
```

Forma <|-- Piramide : 3D

hide circle

hide members

@enduml

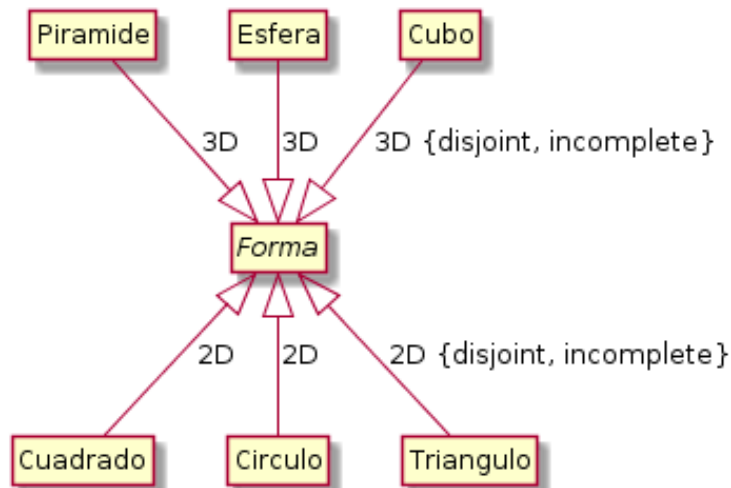


Figura 41.

Como ha visto, los conjuntos de generalización son un concepto de análisis que le permiten dividir un conjunto de subclases. Cuando se refiere a implementación, ninguno de los lenguajes orientados a objetos comúnmente utilizados soporta directamente los conjuntos de generalización. En implementación, los conjuntos de generalización se ignoran o se resuelven en una nueva capa en la jerarquía de herencia, ya que existe un beneficio en hacerlo así. Si se examina el modelo de análisis en la Figura 41, puede imaginar que puede haber algunos atributos u operaciones comunes a todas las 2D y 3D. Esto le proporciona una base para resolver los conjuntos de generalización en nuevas clases en la jerarquía de herencia, como se ilustra en la Figura 42.

@startuml

abstract class Forma

abstract class 2D

Forma <|-- 2D

class Cuadrado

class Circulo

class Triangulo

2D <|-- Cuadrado

2D <|-- Circulo

2D <|-- Triangulo : {disjoint, incomplete}

abstract class 3D

Forma <|-- 3D

class Cubo


```

class Esfera
class Piramide
3D <|--up- Cubo : {disjoint, incomplete}
3D <|--up- Esfera
3D <|--up- Piramide
hide circle
hide members
@enduml

```

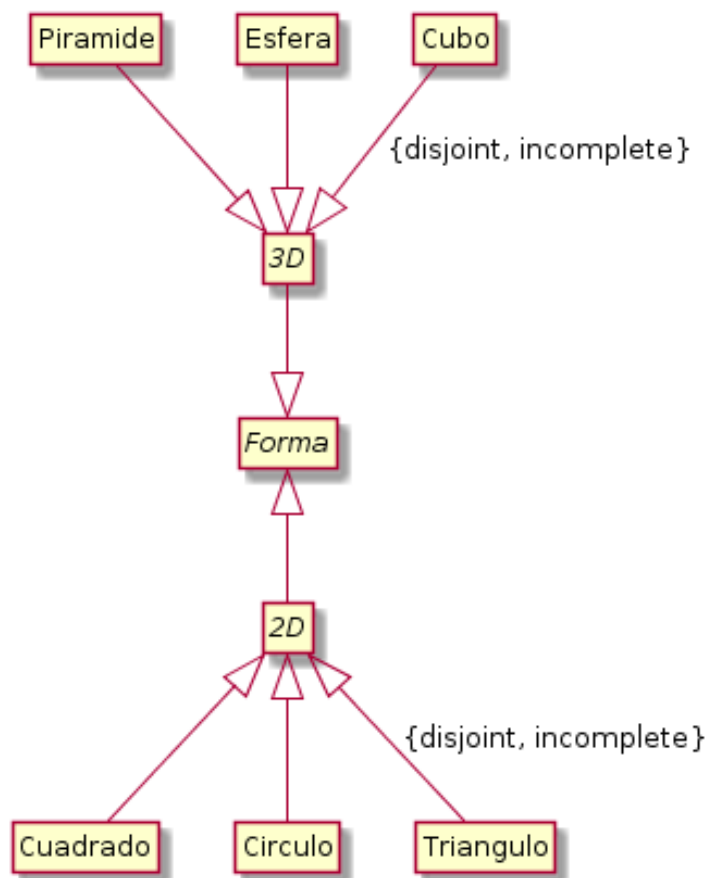


Figura 43.

Clases de análisis - Clases de diseño

El análisis trata sobre modelar lo que el sistema debería hacer. El diseño es sobre modelar cómo el comportamiento se puede implementar.

¿Por qué puede una clase de análisis convertirse en una o más clases de diseño o interfaces? Una clase de análisis se especifica a un nivel muy alto de abstracción. No se preocupa por el conjunto completo de atributos y el conjunto de operaciones es solamente un esbozo que captura los servicios clave ofrecidos por la clase.

Cuando mueve esta clase al diseño, debe especificar totalmente todas las operaciones y atributos, por lo que es bastante común encontrar que la clase se ha hecho muy grande. Si esto sucede, debería desglosarla en una o más clases más pequeñas. Recuerde que siempre

debería tratar de diseñar clases que son pequeñas unidades cohesivas que realizan una o dos tareas realmente bien. Debería evitar a toda costa el tipo de clase "navaja suiza" que trata de hacerlo todo.

El método elegido de implementación determina el grado de totalidad que necesita en las especificaciones de la clase de diseño. Si el modelo de la clase de diseño se proporcionara a programadores que lo utilizarán como una guía para escribir código, entonces las clases de diseño solamente necesitan estar lo suficientemente completas para permitirles realizar esa tarea de forma eficaz. Esto depende de la habilidad que tengan los programadores y lo bien que entiendan los ámbitos del problema y la solución; tiene que averiguar esto para su proyecto en particular.

Sin embargo, si trata de generar código desde las clases de diseño con una herramienta de modelado apropiada, las especificaciones de clase de diseño deben estar completas en todos los aspectos ya que un generador, a diferencia de un programador, no puede llenar los huecos. Con las clases de análisis está tratando de capturar el comportamiento requerido del sistema sin preocuparse de cómo se va a implementar este comportamiento. Con las clases de diseño tiene que especificar exactamente cómo cada clase completará sus responsabilidades. En la Figura 44 se muestra un sencillo ejemplo de una clase de análisis CuentaBancaria refinada durante el diseño (por cuestiones de trazabilidad, ambas clases fueron vinculadas por una relación de dependencia, con el estereotipo <<trace>>)de. Como vio previamente, una operación en una clase de análisis es una especificación lógica de alto nivel de una pieza de funcionalidad ofrecida por una clase. En las clases de diseño correspondientes, cada operación de clase de análisis se convierte en una o más operaciones detalladas que se pueden implementar como código fuente. Por lo tanto, una operación de análisis de alto nivel se puede resolver en una o más operaciones de diseño implementables. Estas operaciones detalladas a nivel de diseño se conocen como métodos.

@startuml

```
class "CuentaBancaria" as ctaAnalisis {
    nombre
    numero
    saldo
    depositar()
    retirar()
    calcularInteres()
}
class "CuentaBancaria" as ctaDisenio {
    - nombre : String
    - numero : String
    - saldo : double = 0
    + CuentaBancaria(nombre:String, numero: String)
    + depositar(m: double): void
    + retirar(m: double): boolean
```

```

+ calcularInteres(): double
+ obtenerNombre(): String
+ establecerNombre(n: String): void
+ obtenerNumero(): String
+ establecerNumero(n: String): void
+ obtenerSaldo(): double
}
ctaAnálisis <.. ctaDiseño : <<trace>>
hide circle
@enduml

```

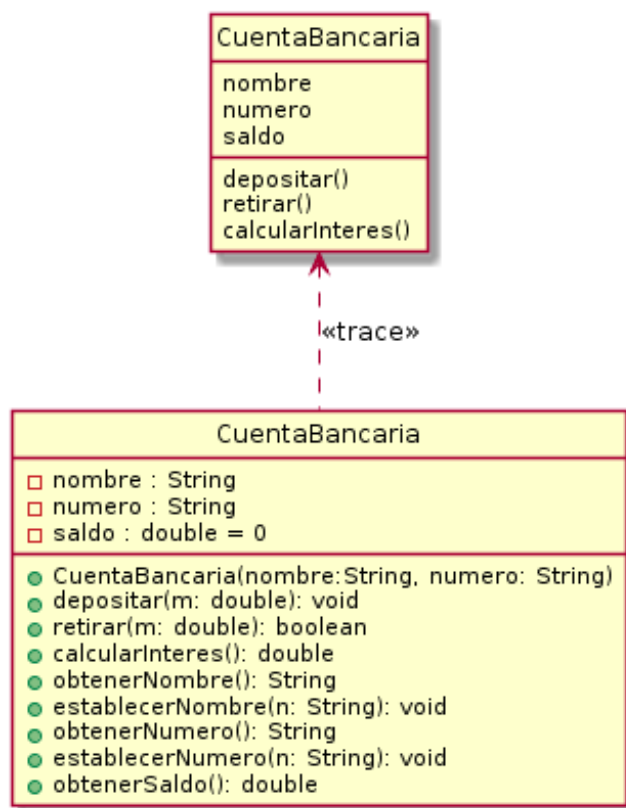


Figura 44.

Características de una clase

Previamente se analizó qué hace una buena clase de análisis, a continuación se explicitarán características que debe reunir toda clase incluida en los modelos:

- Completa y suficiente
- Sencilla
- Alta cohesión
- Bajo acoplamiento

Completa y suficiente

Las operaciones públicas de una clase definen un contrato entre la clase y los clientes de esa clase. Igual que un contrato de negocios es importante que este contrato sea claro, bien definido y aceptable por todas las partes.

La característica de totalidad trata sobre proporcionar a los clientes de una clase lo que podrían esperar. Los clientes realizarán asunciones del nombre de la clase sobre el conjunto de operaciones que debería poner disponible. Cuando nombra una clase y describe su semántica, los clientes de la clase inferirán de esta información qué operaciones se deberían encontrar disponibles. Por ejemplo, una clase CuentaBancaria que proporciona una operación retirar (...) también se esperaría que tuviera una operación depositar (...). Si diseña una clase como un CatalogoProducto, cualquier cliente esperaría razonablemente que esta clase le permitiera añadir, eliminar y encontrar productos en el catálogo. Esta semántica está presupuesta por el nombre de la clase. Por lo tanto, esta característica trata sobre asegurarse de que las clases satisfacen todas las expectativas razonables del cliente.

La suficiencia, por otro lado, es asegurarse de que todas las operaciones de la clase están centradas en realizar la finalidad de la clase. Una clase nunca debería sorprender a un cliente. Debería contener exactamente el conjunto esperado de operaciones y nada más. Por ejemplo, un típico error de principiantes es tomar una clase sencilla como CuentaBancaria y luego añadir operaciones para procesar tarjetas de crédito o administrar pólizas de seguro. La suficiencia trata sobre mantener la clase de diseño lo más sencilla y centrada posible.

La regla de oro para la totalidad y suficiencia es que una clase debería hacer lo que los usuarios de la clase esperan, ni más ni menos.

Sencillez

Las operaciones se deberían diseñar para ofrecer un solo servicio sencillo. Una clase no debería ofrecer múltiples formas de realizar lo mismo ya que esto es confuso para los clientes de la clase y puede llevar a problemas de coherencia y cargas de mantenimiento.

Por ejemplo, si una clase CuentaBancaria tiene una operación sencilla para realizar un solo depósito, no debería tener una operación más compleja que realice dos o más depósitos. Esto es porque puede conseguir el mismo efecto por la aplicación repetida de la operación sencilla. Su objetivo es que las clases deberían siempre poner disponible el conjunto de operaciones más sencillo y pequeño posible.

Aunque la sencillez es una buena regla, existen ocasiones cuando se puede relajar. Una razón común para relajar la restricción de sencillez es mejorar el rendimiento. Por ejemplo, si hubiera un aumento suficiente de rendimiento al realizar depósitos bancarios en un batch, en lugar de individualmente, entonces podría flexibilizar la restricción de sencillez para permitir que una clase CuentaBancaria tenga una operación depositar (...) más compleja que gestione varias transacciones a la vez. Sin embargo, su punto de partida en diseño debería ser siempre el conjunto de operaciones más sencillo posible. Solamente debería añadir complejidad al relajar la sencillez si existe un caso demostrado para hacerlo.

Alta cohesión

Toda clase debería modelar un solo concepto abstracto y debería tener un conjunto de operaciones que soporten el propósito de la clase; esto es cohesión. Si una clase necesita tener muchas responsabilidades diferentes, puede crear clases de ayuda para implementar

algunas de éstas. La clase principal puede entonces delegar responsabilidades a sus clases de ayuda.

La cohesión es una de las características más deseables de una clase. Las clases cohesivas son por lo general fáciles de entender, reutilizar y mantener. Una clase cohesiva tiene un pequeño conjunto de responsabilidades que están íntimamente relacionadas. Toda operación, atributo y asociación de la clase está específicamente diseñado para implementar este pequeño conjunto de responsabilidades.

Por ejemplo, en un sistema de ventas encontramos que existen una clase HotelBean, una clase CocheBean y una clase HotelCocheBean (los "beans son Enterprise JavaBeans (EJBs)). El HotelBean es responsable de vender estadias en hoteles, el CocheBean de vender alquiler de autos y el HotelCocheBean de vender un paquete de alquiler de auto con estadia en hotel. Claramente este modelo es erróneo desde varias perspectivas:

- Las clases están mal nombradas, EstadiaHotel y AlquilerCoche serían nombres mucho mejores.
- El sufijo "Bean" es innecesario ya que simplemente hace referencia a un detalle específico de implementación.
- La clase HotelCocheBean tiene muy mala cohesión, tiene dos responsabilidades principales (vender estadias de hotel y vender alquiler de coche) que ya se llevan a cabo por otras dos clases.

Desde una perspectiva de cohesión, HotelBean y CocheBean son bastante posibles (siempre y cuando se renombren) pero HotelCocheBean es absurdo.

Bajo acoplamiento

Una clase determinada debería estar asociada con suficientes clases para permitirle realizar sus responsabilidades y solamente debería asociar clases si existe un verdadero vínculo semántico entre ellas; esto es bajo acoplamiento.

Uno de los errores comunes de los modeladores orientados a objetos principiantes es conectar todo de cualquier forma. De hecho, el acoplamiento es su peor enemigo en el modelado de objetos y debería ser proactivo tratando de limitar las relaciones entre clases para minimizar el acoplamiento tanto como pueda.

Un modelo de objeto muy acoplado es el equivalente de "código spaghetti" del mundo no orientado a objetos y llevará a un sistema que es incomprensible y no se puede mantener. Encontrará que los sistemas orientados a objetos muy acoplados a menudo resultan de proyectos en los que no existe actividad formal de modelado, donde el sistema simplemente tiene permitido evolucionar con el tiempo.

Si es un modelador principiante, debe tener cuidado de no realizar conexiones entre clases simplemente porque una clase tenga cierto código que otra clase pudiera utilizar. Este es el peor tipo de reutilización ya que sacrifica la integridad arquitectónica del sistema por un pequeño ahorro en tiempo de desarrollo. De hecho, tiene que pensar detenidamente todas las asociaciones entre clases.

Por supuesto, cierto acoplamiento es bueno y deseable. Un alto acoplamiento dentro de un subsistema es generalmente aceptable ya que esto indica una alta cohesión dentro del componente. Solamente compromete la arquitectura cuando el acoplamiento es entre subsistemas y tiene que tratar de reducir este tipo de acoplamiento.

Agregación y composición

Puede hacerse una idea de las diferencias semánticas entre los dos tipos de agregación al pensar en ejemplos del mundo real.

- Agregación: Es un tipo de relación todo-parte; un ejemplo podría ser un ordenador y sus periféricos.
- Composición: Este es un tipo muy fuerte de relación todo-parte entre objetos; es como un árbol y sus hojas.

Un ordenador está poco relacionado con sus periféricos. Estos periféricos pueden estar o no, se pueden compartir entre ordenadores y no son propiedad de ningún ordenador en particular; esto es agregación. Por otro lado, un árbol está íntimamente relacionado con sus hojas. Las hojas son propiedad exactamente de un árbol; no se pueden compartir entre árboles, y cuando el árbol muere, las hojas van con él; esto es composición. Sería de mucha utilidad mantener estas sencillas analogías en mente a medida que estudia la semántica detallada de agregación y composición.

Semántica de agregación

La agregación es un tipo de relación todo-parte en la que el conjunto es compo de muchas partes. En una relación todo-parte, un objeto (el todo) utiliza los servicios de otro objeto (la parte). Como tal, el todo tiende a ser la parte dominante y controladora de la relación, mientras que la parte tiende a servir peticiones desde el todo y por lo tanto es más pasiva. Considere el ejemplo de agregación de la Figura 45.

```
@startuml
class Ordenador
class Impresora
Ordenador "0..1" o-- "*" Impresora
note right on link
    agregación
end note
hide circle
hide members
@enduml
```

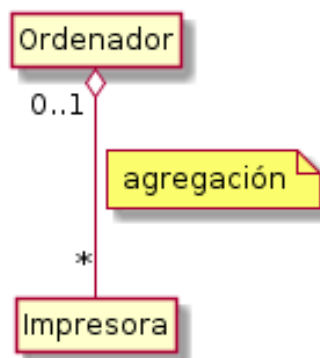


Figura 45.

Puede ver que:

- Un Ordenador se puede anexar a 0 o más Impresoras.
- En cualquier momento en el tiempo, una Impresora se conecta a 0 o 1 Ordenador.
- Con el tiempo, muchos Ordenadores pueden utilizar una Impresora dada
- La Impresora puede existir incluso si no hay Ordenadores anexados.
- La Impresora es independiente del Ordenador.

Podemos resumir la semántica de la agregación de la siguiente forma:

- El conjunto puede existir algunas veces independientemente de las partes algunas veces no.
- Las partes pueden existir independientemente del conjunto.
- Es posible tener propiedad compartida de las partes por varios conjuntos.

En UML no necesariamente se considera a la agregación como transitiva. Sin embargo, diversos autores, como Arlow y Neustadt (2006), Rumbaugh y colab. (2007), sí la consideran.

El ejemplo en la Figura 46 muestra otro ejemplo típico de agregación. Puede modelar un ordenador (el todo) como un conjunto de partes. Estas partes están bastante poco relacionadas con el todo ya que son intercambiables con otros ordenadores y se pueden compartir con otros ordenadores, por lo que puede utilizar semántica de agregación en su modelo. El modelo dice que un ordenador se podría pensar como un conjunto de las siguientes partes: un Ratón, un Teclado, una CPU, un Monitor y dos Altavoces. La CPU se puede modelar como un conjunto de varios componentes de hardware como RAM, DiscosDuros, etc.

@startuml

class Ordenador

class Raton

class Teclado

class CPU

class Monitor

class Altavoz

Ordenador "0..1" o-- "1" Raton

Ordenador "0..1" o-- "1" Teclado

Ordenador "0..1" o-- "1" CPU

Ordenador "0..1" o-- "1" Monitor

Ordenador "0..1" o-- "2" Altavoz

class RAM

class DiscoDuro

class TarjetaSonido

class TarjetaGrafica

CPU "0..1" o-- "*" RAM

CPU "0..1" o-- "1..*" DiscoDuro

CPU "0..1" o-- "1" TarjetaSonido

```

CPU "0..1" o-- "1" TarjetaGrafica
Monitor "1" -- "1" TarjetaGrafica :> ConectadoA
Altavoz "2" -- "1" TarjetaSonitdo :> ConectadoA
hide circle
hide members
@enduml

```

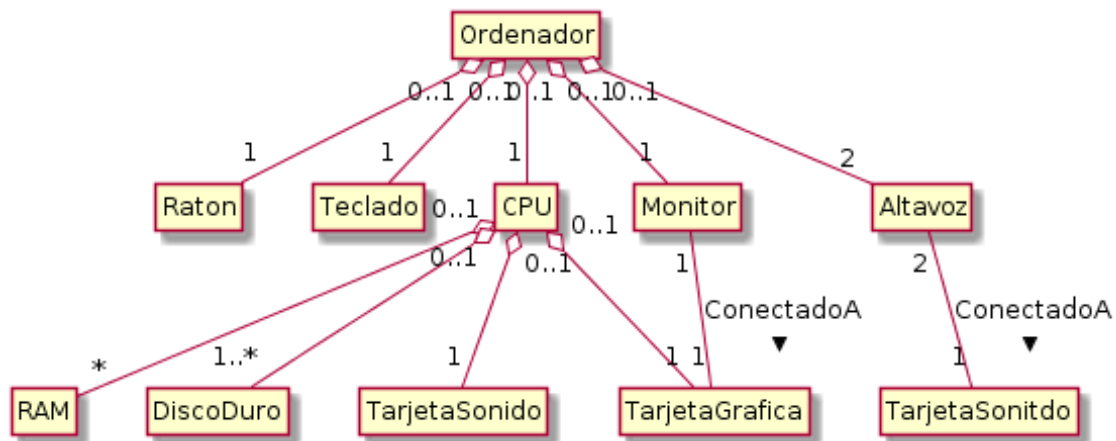


Figura 46.

Semántica de composición

La composición es una forma más fuerte de agregación y tiene semántica similar (pero más restringida). Al igual que la agregación, es una relación todo-parte. La diferencia clave entre agregación y composición es que en composición las partes no tienen vida independiente fuera del todo. Además, en composición, cada parte pertenece al menos a un y sólo a un todo, mientras que en agregación una parte se puede compartir entre los todos.

En el ejemplo en la Figura 47, los objetos Botón no tienen existencia independiente aparte del objeto Ratón que lo posee. Si destruye el objeto Ratón, puede destruir sus objetos Botón ya que son una parte integral de éste. Todo objeto Botón puede pertenecer exactamente a un objeto Ratón. Esto es como las hojas y los árboles; la vida de la hoja está determinada por la vida del árbol, y la hoja puede solamente pertenecer a exactamente un árbol. Podemos resumir la semántica de composición de la siguiente forma:

- Las partes pueden solamente pertenecer a un conjunto de cada vez; no existe posibilidad de propiedad compartida de una parte.
- El conjunto tiene responsabilidad única para la disposición de todas sus partes; esto significa la responsabilidad para su creación y destrucción.
- El conjunto puede liberar partes, siempre y cuando la responsabilidad para ellas se asuma por otro objeto.
- Si se destruye el conjunto, debe destruir todas sus partes o pasar la responsabilidad a algún otro objeto.

Puesto que el conjunto tiene responsabilidad única para el ciclo de vida y disposición de sus partes, cuando crea un conjunto, el objeto conjunto creará sus partes. De forma similar,

cuando destruye un conjunto, el conjunto debe destruir todas sus partes u organizarlas para ser adoptadas por otro conjunto.

En agregación es posible definir jerarquías de árbol y redes, en cambio, con composición, solamente se puede tener jerarquías de árbol. Esto es porque en composición, un objeto parte solamente puede ser parte de un conjunto en cualquier momento en el tiempo.

```
@startuml
class Raton
note right
    Compuesto/Todo
end note
class Boton
note right
    Componente/Parte
end note
Raton "1" *-- "1..4" Boton
hide circle
hide members
@enduml
```

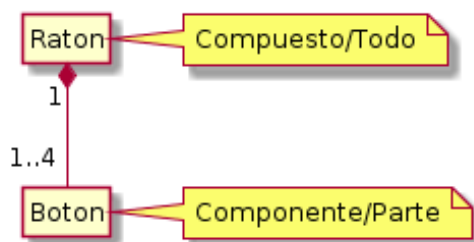


Figura 47.

En la Figura 48, un motor puede ser opcionalmente parte de un auto o de una lancha. No puede ser parte de los dos al mismo tiempo (por reglas de la composición, es un XOR).

```
@startuml
class Motor
class Lancha
class Auto
Auto "0..1" *-- "1" Motor
Lancha "0..1" *-- "1" Motor
hide circle
hide members
@enduml
```

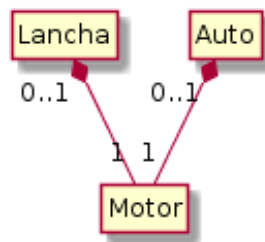


Figura 48.

Composición y atributos

Cuando piensa en la semántica de la composición, debería ver que es muy similar a la semántica de atributos. Ambas tienen ciclos de vida que están controlados por sus propietarios, y ambas no tienen existencia independiente fuera de sus propietarios. De hecho, los atributos son exactamente equivalentes a una relación de composición entre la clase conjunto y la clase del atributo. ¿Por qué, entonces, necesitamos dos formas de expresar lo mismo? Existen dos razones:

- Los atributos pueden ser tipos de datos primitivos. Ciertos lenguajes híbridos orientados a objetos como C++ y Java tienen tipos primitivos como `int` y `double` que no son clases. Ahora, puede modelarlos como clases estereotipadas <<primitive>>, pero solamente saturará el modelo. Estos tipos primitivos siempre se deberían modelar como atributos.
- Existen ciertas clases de utilidad como `Time`, `Date` y `String` que se utilizan de forma generalizada. Si tuviera que modelar cada utilización de una de estas clases por una relación de composición a la propia clase, pronto sus modelos estarán totalmente ocultos. Es mejor modelar clases como éstas como atributos.

El hecho es que si tiene un tipo primitivo o una clase de utilidad, o incluso una clase que no es muy interesante o de utilidad para mostrarse explícitamente en el modelo, debería considerar utilizar un atributo en lugar de una relación de composición.

Agregación vs, herencia

Herencia es una técnica muy potente. Sin embargo, los diseñadores y programadores orientados a objetos principiantes a menudo abusan de ello. Debería darse cuenta que la herencia tiene ciertas características no deseables:

- Es la forma mayor de acoplamiento posible entre dos o más clases.
- La encapsulación es débil dentro de una jerarquía de clase. Los cambios en la clase base provocan cambios en las subclases. Esto lleva a lo que se conoce como el problema "clase base frágil" donde los cambios en las clases base tienen un importante impacto sobre otras clases en el sistema.
- Es un tipo muy inflexible de relación. En todos los lenguajes orientados a objetos comúnmente utilizados, es posible actualizar las jerarquías de agregación y composición en tiempo de ejecución al crear y destruir relaciones, pero las jerarquías de herencia permanecen fijas. Esto hace que sea el tipo más inflexible de relación entre clases.

El ejemplo en la Figura 49 es una solución típica de principiante al problema de modelar roles en una organización. A primera vista, parece bastante posible, pero tiene problemas. Considere este caso: el objeto juan es de tipo Programador y desea promocionarlo para que sea del tipo Director. ¿Cómo puede hacer esto? Ha visto que no puede cambiar la clase de juan en tiempo de ejecución y por lo tanto, la única forma en que puede conseguir la promoción es crear un nuevo objeto Director (denominado juan : Director) copiar todos los datos relevantes del objeto juan : Programador y luego eliminar el objeto juan : Programador para mantener coherencia en la aplicación. Esto es complejo y en absoluto como funciona el mundo real, cuando está mal modelado.

```
@startuml
class Empleado
class Director
class Programador
Empleado <|-- Director
Empleado <|-- Programador
object "<u>juan:Programador</u>" as juan
Programador <.. juan : <<instantiate>>
hide circle
hide members
@enduml
```

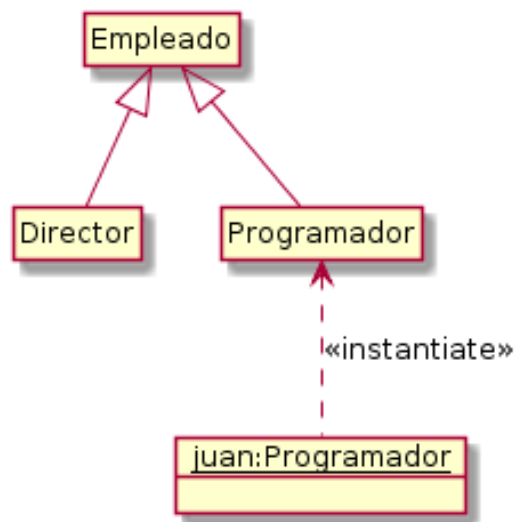


Figura 49.

Por supuesto, existe un error semántico fundamental en el modelo en la Figura 49. ¿Es un empleado su trabajo, o bien es que un empleado tiene un trabajo? Esta pregunta nos lleva a la solución del problema que se muestra en la Figura 50.

```
@startuml
class Empleado
```

```

class Trabajo
class Director
class Programador
Empleado "*" o-- "*" Trabajo
Trabajo <|-- Director
Trabajo <|-- Programador
object "<u>juan:Empleado</u>" as juan
object "<u>:Director</u>" as director
object "<u>:Programador</u>" as programador
Empleado <.. juan : <<instantiate>>
Director <.. director : <<instantiate>>
Programador <.. programador : <<instantiate>>
juan o-- programador
note right on link
    cambie este vínculo
    en tiempo de ejecución
    para promocionar a juan a Director
end note
hide circle
hide members
@enduml

```

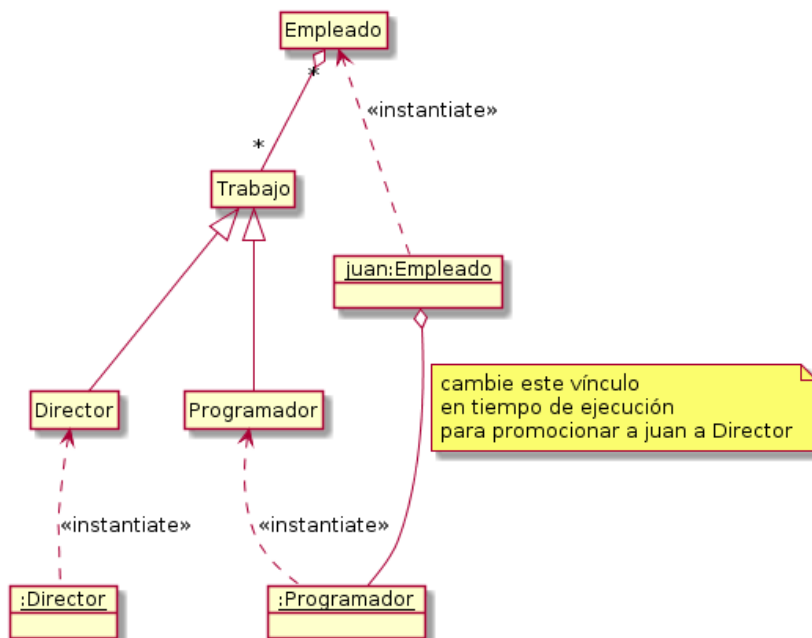


Figura 50.

Utilizando agregación obtiene la semántica correcta, un Empleado tiene un Trabajo. Con este modelo más flexible, los Empleados también pueden tener más de un Trabajo, si fuera necesario.

Hemos conseguido un modelo mucho más flexible y semánticamente correcto al reemplazar la herencia con agregación como el mecanismo para asignar trabajos a empleados. Existe un principio general importante aquí; las subclases siempre deberían representar "es tipo de" en lugar de "es rol desempeñado por". Cuando pensamos en la semántica de negocio de empresas, empleados y trabajos, está claro que un trabajo es un "rol desempeñado por" un empleado y no indica realmente "un tipo de" empleado.

Como tal, la herencia es la opción errónea para modelar este tipo de relación de negocio. Por otro lado, existen muchos tipos de trabajo en una empresa. Esto indica que una jerarquía de herencia de trabajos (que parten de la clase base abstracta Trabajo) es probablemente un buen modelo.

Herencia múltiple

Algunas veces es posible que desee heredar de más de un padre. Esto es herencia múltiple y no se soporta en todos los lenguajes orientados a objetos. Por ejemplo, Java y C# solamente permiten herencia sencilla. En la práctica, esta ausencia de soporte para herencia múltiple no es un problema y siempre se puede reemplazar por herencia sencilla y delegación.

Aunque la herencia múltiple, algunas veces ofrece la solución más elegante a problema de diseño, solamente se puede utilizar si el lenguaje de implementación de destino lo soporta. Los puntos importantes sobre la herencia múltiple son los siguientes:

- Todas las clases padre implicadas deben estar separadas semánticamente. Si hay algún solapamiento en semántica entre las clases base, existe la posibilidad de interacciones imprevistas entre ellas. Esto podría llevar a un comportamiento extraño en la subclase. Decimos que las clases base deben ser todas ortogonales.
- Los principios de "es tipo de" y "sustitución" deben aplicarse entre la subclase y todas sus superclases.
- Normalmente, las superclases no deberían tener ningún padre en común. De lo contrario, tiene un ciclo en la jerarquía de herencia y puede haber múltiples rutas en donde las mismas características se podrían heredar de clases más abstractas. Los lenguajes que soportan herencia múltiple (como C++) tienen formas específicas, dependientes del lenguaje, de resolver ciclos en la jerarquía de herencia.

Resumen de la notación UML en PlantUML - PLanttext

Notación de objeto UML – PlantUML PLanttext

constructor Diagrama de Objetos	PlantUML - planttext https://www.planttext.com/ http://plantuml.com/
diagrama de objetos	@startuml ' elementos del diagrama de objetos ' @enduml

objeto Objeto identificado con sólo el nombre de la clase	object " <u>:NombreDeClase</u> " as unObjetoDeNombreDeClase { /' propiedades del objeto /' } 'luego, es posible emplear el objeto referenciando unObjetoDeNombreDeClase
objeto Objeto identificado con sólo el nombre de objeto	object "<u>nombreDeObjeto</u>" as nombreDeObjeto { /' propiedades del objeto /' } 'luego, es posible emplear el objeto referenciando nombreDeObjeto
objeto Objeto identificado con el nombre del objeto concatenado con el nombre de clase, separado por dos puntos	object "<u>nombreDeObjeto:NombreDeClase</u>" as nombreDeObjeto { /' propiedades del objeto /' } 'luego, es posible emplear el objeto referenciando nombreDeObjeto
atributo: TipoDeAtributo = valor	atributo: TipoDeAtributo = valor
link entre objeto1 y objeto2	objeto1 -- objeto2
link unidireccional desde objeto1 a objeto2	objeto1 --> objeto2

Por ejemplo, para indicar que un banco (instancia de Banco) posee la cuentaDeNicolas (instancia de Cuenta) y la cuenta de Nicolás posee los atributos numeroDeCuenta = "12345", propietario = "Nicolás", y saldo= 300,00, se puede definir el siguiente diagrama de objetos:

@startuml

object "<u>Banco</u>" as unBanco

object "<u>cuentaDeNicolas:Cuenta</u>" as cuentaDeNicolas {

numeroDeCuenta = "12345"

propietario = "Nicolás"

saldo = 300,00

}

unBanco -- cuentaDeNicolas

@enduml

Notación de clases UML – PlantUML Planttext

constructor Diagrama de Clases	PlantUML - planttext https://www.planttext.com/ http://plantuml.com/
diagrama de clases	@startuml /' elementos del diagrama de clases /' hide circle @enduml
clase	class NombreDeClase { /' propiedades de la clase (atributos) operaciones de la clase /' } 'luego, es posible emplear la clase referenciando NombreDeClase
clase abstracta	abstract class NombreDeClase
atributo: Tipo	atributo: Tipo
atributo: Tipo [multiplicidad]	atributo: Tipo [multiplicidad]
atributo a nivel de clase	{static} atributo: Tipo
operacion()	operacion()
operacion(param1: TipoParam1, ..., paramN: TipoParamN): TipoRetorno	operacion(param1: TipoParam1, ..., paramN: TipoParamN): TipoRetorno
operación a nivel de clase	{static} operación()
operación abstracta	{abstract} operación()
asociación entre Clase1 y Clase2	Clase1 -- Clase2
asociación entre Clase1 y Clase2 denominada NombreDeAsociacion	Clase1 -- Clase2 : NombreDeAsociacion
asociación entre Clase1 y Clase2 denominada NombreDeAsociacion e indicando que se lee el nombre desde Clase1 hacia Clase2	Clase1 -- Clase2: NombreDeAsociacion >
asociación entre Clase1 y Clase2 con multiplicidad 0..1 del lado Clase1 y multiplicidad * en el extremo de Clase2	Clase1 "0..1" -- "*" Clase2

asociación entre Clase1 y Clase2 con rol clase1 del lado Clase1 y rol clase2 en el extremo de Clase2	Clase1 "clase1" -- "clase2" Clase2
asociación entre Clase1 y Clase2 con multiplicidad 0..1 y rol clase1 del lado Clase1 y multiplicidad * y rol clase2 en el extremo de Clase2	Clase1 "clase1 0..1" -- "clase2 *" Clase2
asociación unidireccional desde Clase1 hacia Clase2	Clase1 --> Clase2
asociación de clase AC1 entre Clase1 y Clase2	Clase1 -- Clase2 (Clase1, Clase2) .. AC1 class AC1 { /' propiedades de la clase AC1 (atributos) operaciones de la claseAC1 '/ }
ClaseMasGeneral generaliza/especializadaPor ClaseMasEspecificas	ClaseMasGeneral < -- ClaseMasEspecificas
Relación de composición Todo asociaciónDeComposición Parte	Todo *-- Parte
Relación de agregación Todo asociaciónDeAgregación Parte	Todo o-- Parte
Relación de dependencia Clase1 depende de Clase2	Clase1 ..> Clase2
Enumeración TipoEnum con valores posibles: valor1, valor2	enum TipoEnum { valor1 valor2 }

Por ejemplo, para representar un Banco asociado a un conjunto de cuentas (Cuenta, clase abstracta). Toda Cuenta está asociada a un Banco. Una Cuenta puede ser CajaDeAhorro o CuentaCorriente. En cuenta se definen las operaciones abstractas retirar(cantidad) y calcularIntereses(). Además, se define la operación depositarCantidad(), operación que es redefinida en CajaDeAhorro.

@startuml

```

abstract class Cuenta {
    {abstract} retirar(cantidad)
    {abstract} calcularIntereses()
    depositarCantidad()
}

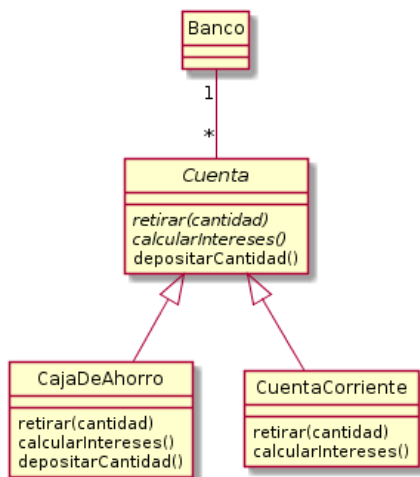
```



```

class CajaDeAhorro {
    retirar(cantidad)
    calcularIntereses()
    depositarCantidad()
}
class CuentaCorriente {
    retirar(cantidad)
    calcularIntereses()
}
Cuenta <|-- CajaDeAhorro
Cuenta <|-- CuentaCorriente
class Banco
Banco "1" -- "*" Cuenta
hide circle
@enduml

```



Bibliografía

- Arlow, J., I. Neustadt. Enterprise patterns and MDA: Building better software with Archetype patterns and UML, Addison-Wesley, 2004.
- Arlow, J., I. Neustadt. UML 2.0. Anaya, 2006.
- Meyer, B., Object oriented software construction, Prentice Hall, 1997.
- Rumbaugh, J., I. Jacobson, G. Booch. El lenguaje unificado de modelado. Manual de Referencia. Segunda Edición. Pearson, 2007.